Universidad Mariano Gálvez de Guatemala, Boca del Monte, Guatemala

Curso: Sistemas Operativos

Catedrático: Ing. Julio Gómez

Actividad:

REPOSITORIO HIT HUB

SIMULADOR DE FUNCIONES DEL KERNEL DE UN SISTEMA OPERATIVO

Informe de Proyecto Tecnológico — Proyecto Final

Integrantes:

Dalila Betsabeth García Acabal
7690-23-3074
Angel Steve Morejón Martínez
7690-23-21973
Edgar Yovany García Luis
7690 23 13075
Angel Aurelio Tzi Cucul
7690-23-3711

Índice

- Resumen Ejecutivo
- 1. Introducción del problema a resolver
- 2. Identificación del problema
- 3. Justificación
- 4. Objetivos (estándar SMART): 4.1 General, 4.2 Específicos
- 5. Marco Teórico (incluye análisis teórico de Round Robin)
- 6. Metodología: 6.1 Diagnóstico y Diseño (modelo de bloques); 6.2 Planeación (WBS, RACI, canal de comunicación); 6.3 Desarrollo y ejecución (stack tecnológico, requerimientos, construcción y pruebas del prototipo)
- 7. Descripción del proceso de evaluación y ajustes
- 8. Declaración de Autoría y Honestidad Académica
- 9. Conclusiones y Recomendaciones
- 10. Referencias Bibliográficas

Resumen Ejecutivo

Este informe documenta el diseño, la construcción y la validación de un **Simulador de Funciones del Kernel de un Sistema Operativo**, desarrollado en Java como proyecto final del curso Sistemas Operativos I. El simulador no interactúa con el hardware real: modela mediante estructuras de datos los mecanismos internos que un núcleo debe resolver, y los expone mediante una interfaz de línea de comandos (CLI) con menú de texto y, de forma complementaria, una interfaz gráfica Swing llamada KernelLab que opera sobre el mismo motor.

El sistema integra cinco componentes de forma modular. Primero, un **Bloque de Control de Proceso (PCB)** con PID, estado, tamaño en memoria, ráfaga total y ráfaga restante. Segundo, un **planificador Round Robin** con quantum fijo q = 3 y cambio de contexto simulado. Tercero, un **administrador de memoria** de 100 MB con asignación contigua dinámica (First-Fit) que rechaza con un error Out of Memory todo proceso que no encuentre un hueco suficiente. Cuarto, un **semáforo binario** con operaciones explícitas wait() y signal() y cola de espera propia que protege una variable global compartida. Quinto, un **gestor de dos recursos exclusivos** con detección de interbloqueo mediante grafo de espera y resolución por aborto forzoso de un proceso.

La validación se realizó con cuatro casos de prueba automáticos y reproducibles. El rechazo por falta de RAM se comprobó con solicitudes de 40, 40, 30, 20 y 1 MB, de las cuales dos fueron rechazadas. Round Robin, con ráfagas 7, 4 y 5, produjo tiempos de espera de 9, 9 y 10 unidades (promedio 9.33), idénticos al cálculo teórico. El semáforo corrigió una condición de carrera: sin él el contador global terminó en 2 en lugar de 4; con él, en 4. Finalmente, el deadlock entre dos procesos fue detectado y resuelto abortando uno de ellos, tras lo cual el otro finalizó con normalidad.

El código fuente está modularizado, comentado y alojado en un repositorio de GitHub. El resultado cumple los requerimientos del enunciado y constituye una base didáctica para comprender cómo un sistema operativo administra procesos, memoria, sincronización e interbloqueos.

1. Introducción del problema a resolver

Un sistema operativo es el software que administra los recursos del computador y ofrece a los programas un entorno uniforme para ejecutarse. En su núcleo, o kernel, se concentran las decisiones más delicadas: qué proceso usa la CPU y por cuánto tiempo, cómo se reparte la memoria física, cómo se evita que dos procesos corrompan datos compartidos y qué hacer cuando varios procesos se bloquean entre sí esperando recursos que ninguno liberará.

Estos mecanismos suelen estudiarse de forma teórica, pero su comportamiento solo se comprende de verdad al verlo operar: observar cómo un quantum se agota y el proceso regresa al final de la cola, cómo una petición de memoria falla por falta de un hueco contiguo, cómo se pierde una actualización cuando no existe exclusión mutua, o cómo dos procesos quedan detenidos para siempre en una espera circular.

El proyecto aborda esa necesidad construyendo un **simulador del kernel** que reproduce fielmente la lógica de cuatro funciones fundamentales: planificación de procesos, gestión de memoria, concurrencia con sincronización e interbloqueo. El simulador trabaja con estructuras de datos y no con hardware real, lo que permite experimentar con seguridad, repetir escenarios y verificar resultados contra los cálculos teóricos vistos en clase.

2. Identificación del problema

El problema que resuelve este proyecto puede enunciarse así:

> *¿Cómo representar y demostrar, mediante un programa interactivo y verificable, el funcionamiento lógico de las funciones centrales del kernel de un sistema operativo (planificación Round Robin, asignación de memoria, exclusión mutua con semáforos y manejo de interbloqueo), de modo que el comportamiento teórico pueda observarse, medirse y comprobarse?*

De esa pregunta se derivan los problemas específicos que el simulador debe resolver:

- **Planificación:** repartir la CPU de forma equitativa entre procesos con ráfagas distintas, sin que ninguno monopolice el procesador, y calcular los tiempos de espera y de retorno resultantes.
- **Memoria:** decidir en el momento de crear un proceso si cabe en una RAM limitada a 100 MB y rechazarlo de inmediato, con un error claro, cuando no cabe.
- **Concurrencia:** garantizar que solo un proceso a la vez acceda a una sección crítica y mostrar qué ocurre cuando esa garantía no existe.
- **Interbloqueo:** provocar una espera circular entre dos procesos y dos recursos, detectarla, alertar al usuario y ofrecer una forma de romperla.
- **Robustez:** resistir entradas inválidas o aleatorias (negativos, texto, valores fuera de rango, identificadores inexistentes) sin colapsar, dado que el catedrático realizará una prueba de estrés en vivo.

3. Justificación

La justificación del proyecto se apoya en cuatro razones.

Académica. Los temas de planificación, memoria, sincronización e interbloqueo son el eje del curso. Implementarlos obliga a pasar de la definición a la decisión de diseño: qué estructura de datos usar, en qué orden se encolan los procesos, cuándo se libera un recurso. Esa traducción consolida el aprendizaje mucho más que la lectura.

Técnica. Simular el kernel evita los riesgos y la complejidad de modificar un núcleo real. Un error en el simulador produce un mensaje, no un sistema caído, y cada escenario puede repetirse con exactamente los mismos parámetros, lo cual es indispensable para documentar y verificar resultados.

Didáctica. El simulador sirve como herramienta de demostración: la salida por consola y la interfaz gráfica muestran el diagrama de Gantt, el mapa de memoria, la cola de espera del semáforo y el ciclo del deadlock, de modo que un tercero puede entender el comportamiento sin leer el código.

Formativa. El proyecto exige modularidad, validación de entradas, pruebas documentadas, control de versiones en GitHub y redacción de un informe formal, competencias propias del ejercicio profesional de un ingeniero en sistemas.

4. Objetivos (estándar SMART)

Cada objetivo se formula para ser Específico, Medible, Alcanzable, Relevante y con un Tiempo definido (la fecha de entrega del proyecto).

4.1 Objetivo general

Diseñar, implementar y documentar, para la fecha de entrega del proyecto final, un simulador por línea de comandos del núcleo de un sistema operativo en Java que integre de forma modular los componentes de planificación Round Robin, gestión de memoria de 100 MB, sincronización con semáforo binario y manejo de interbloqueo, y que supere los cuatro casos de prueba exigidos en el enunciado con resultados reproducibles.

4.2 Objetivos específicos

N.º	Objetivo	Indicador de cumplimiento
1	Implementar el PCB (PID, estado, tamaño, ráfaga total y restante) y el planificador Round Robin con q = 3 y cambio de contexto simulado.	El diagrama de Gantt y los tiempos de espera obtenidos para las ráfagas 7, 4 y 5 coinciden al 100 % con el cálculo teórico.
2	Construir un administrador de memoria de 100 MB con asignación contigua dinámica que rechace procesos por desbordamiento.	Toda solicitud mayor que el mayor hueco contiguo es rechazada con el error Out of Memory (caso de prueba 1).
3	Programar un semáforo binario con wait(), signal() y cola de espera que proteja una variable global compartida.	Con semáforo el contador alcanza el valor esperado (4 de 4); sin semáforo se evidencia la pérdida de actualizaciones (2 de 4).
4	Desarrollar la detección de espera circular entre dos recursos y un comando que aborte un proceso para resolverla.	El deadlock es detectado, alertado y eliminado, y el proceso restante termina con éxito (caso de prueba 4).
5	Validar todas las entradas del usuario y soportar una prueba de estrés con parámetros aleatorios.	Ninguna entrada inválida (texto, vacío, negativos, desbordes, PID inexistente) provoca una excepción no controlada.
6	Documentar el proyecto y publicar el código en GitHub.	Informe formal en PDF entregado y repositorio con el código modular y comentado.

5. Marco Teórico

5.1 Kernel y procesos

El **kernel** es la parte del sistema operativo que se ejecuta con los máximos privilegios y gestiona CPU, memoria, dispositivos y comunicación entre procesos. Un **proceso** es un programa en ejecución junto con su estado: contador de programa, registros, memoria asignada y recursos que utiliza.

Para administrar cada proceso, el kernel mantiene un **Bloque de Control de Proceso (PCB)**, una estructura que almacena como mínimo el identificador (PID), el estado, información de planificación y de memoria. En el simulador, el PCB contiene el PID, el estado, el tamaño en memoria, la ráfaga de CPU total y la ráfaga restante, además de campos auxiliares para estadísticas.

5.2 Estados de un proceso

El simulador adopta el modelo clásico de cinco estados:

Estado	Significado
NUEVO	Proceso recién creado que aún no entra a la cola de listos.
LISTO	En la cola de listos, esperando que se le asigne la CPU.
EJECUCIÓN	Usando la CPU durante su turno.
BLOQUEADO	Esperando un semáforo o un recurso.
TERMINADO	Finalizó su ráfaga o fue abortado.

5.3 Planificación Round Robin

Round Robin (RR) es un algoritmo de planificación apropiativo pensado para sistemas de tiempo compartido. Los procesos esperan en una cola FIFO y cada uno recibe la CPU durante un intervalo máximo llamado **quantum**. Si el proceso termina antes, libera la CPU; si agota el quantum sin terminar, ocurre un **cambio de contexto**: se guarda su estado y regresa al final de la cola, y la CPU pasa al siguiente.

Sus ventajas son la equidad y que ningún proceso sufre inanición. Su desempeño depende del quantum: si es muy grande se comporta como FCFS, y si es muy pequeño el costo de los cambios de contexto domina. Este proyecto fija q = 3 unidades de tiempo.

5.3.1 Análisis teórico de los tiempos de espera

Cuando todos los procesos llegan en t = 0, las fórmulas son:

- **Tiempo de retorno** = tiempo de finalización − tiempo de llegada
- **Tiempo de espera** = tiempo de retorno − ráfaga total

Sea el ejemplo de tres procesos P1 (ráfaga 7), P2 (ráfaga 4) y P3 (ráfaga 5), con q = 3. La secuencia de ejecución es:

Gantt: P1 (0–3) | P2 (3–6) | P3 (6–9) | P1 (9–12) | P2 (12–13) | P3 (13–15) | P1 (15–16)

Proceso	Ráfaga	Finalización	Retorno	Espera (retorno − ráfaga)
P1	7	16	16	16 − 7 = 9
P2	4	13	13	13 − 4 = 9
P3	5	15	15	15 − 5 = 10
Promedio			**14.67**	**9.33**

La suma de ráfagas es 7 + 4 + 5 = 16, por lo que la CPU termina en t = 16 sin tiempos muertos. Se producen 4 cambios de contexto, uno cada vez que un proceso agota su quantum sin terminar.

5.4 Gestión de memoria

La memoria principal es un recurso limitado que el sistema operativo reparte entre los procesos. En la **asignación contigua dinámica** (particiones variables) cada proceso recibe un único bloque contiguo del tamaño exacto que solicita. Al liberarse, ese bloque vuelve a ser un hueco, y los huecos adyacentes se fusionan.

Para elegir el hueco se usan políticas como **First-Fit** (el primer hueco suficiente), Best-Fit (el más ajustado) o Worst-Fit (el más grande). Este esquema produce **fragmentación externa**: puede haber memoria libre suficiente en total pero dispersa en huecos pequeños, de modo que una solicitud falla pese a que la suma de los huecos la cubriría. Cuando no existe un hueco adecuado se produce un error de desbordamiento (**Out of Memory**).

5.5 Concurrencia y exclusión mutua

Una **condición de carrera** ocurre cuando el resultado depende del orden en que se intercalan las operaciones de varios procesos sobre un dato compartido. Una instrucción como `global++` se descompone en leer, incrementar y escribir; si dos procesos leen el mismo valor antes de que el otro escriba, una actualización se pierde.

La porción de código que accede al dato compartido es la **sección crítica**, y debe ejecutarse con **exclusión mutua**: solo un proceso a la vez. Dijkstra propuso el **semáforo**, una variable entera protegida cuyo acceso se realiza mediante dos operaciones atómicas. El **semáforo binario** toma valor 1 (libre) o 0

(ocupado): `wait()` bloquea al proceso si el valor es 0 y lo coloca en una cola de espera; `signal()` libera la sección o despierta al primer proceso en espera.

5.6 Interbloqueo (deadlock)

Un **interbloqueo** es una situación en la que un conjunto de procesos queda bloqueado indefinidamente porque cada uno espera un recurso que retiene otro del mismo conjunto. Según Coffman, ocurre cuando se cumplen simultáneamente cuatro condiciones: **exclusión mutua**, **retención y espera**, **no apropiación** y **espera circular**.

El ejemplo clásico usa dos procesos y dos recursos: A retiene el Recurso 1 y solicita el 2, mientras B retiene el 2 y solicita el 1. Para detectarlo se emplea un **grafo de espera** (wait-for), donde existe una arista P → Q si P espera un recurso retenido por Q; **un ciclo en el grafo indica interbloqueo**. Una estrategia de recuperación es **terminar un proceso** del ciclo, lo que libera sus recursos y rompe la espera circular.

6. Metodología

La metodología siguió cuatro etapas: diagnóstico y diseño, planeación, desarrollo y ejecución, y evaluación con ajustes (sección 7). El trabajo fue iterativo: cada módulo se construyó, se probó de forma aislada y luego se integró al Kernel.

6.1 Diagnóstico y Diseño

Diagnóstico. A partir del enunciado oficial se identificaron los cinco componentes obligatorios (procesos con PCB, Round Robin con q = 3, RAM de 100 MB, semáforo binario y deadlock con resolución), las condiciones de entrega (código modular en GitHub, interfaz interactiva por consola, informe formal y demostración en vivo) y el principal riesgo: que el catedrático introduzca parámetros aleatorios durante la presentación. Ese riesgo fijó un criterio de diseño transversal: ninguna entrada llega al núcleo sin ser validada.

Decisiones de diseño.

Tema	Decisión	Razón
Lenguaje	Java	El enunciado permite Python, Java o C++. Java ofrece tipado fuerte, orientación a objetos y la biblioteca gráfica Swing incluida en el JDK, sin dependencias externas.
Memoria	Asignación contigua dinámica con First-Fit	Es una de las estrategias admitidas y hace visible el desbordamiento y la fragmentación externa. Los huecos vecinos se fusionan al liberar.
Sincronización	Semáforo binario con propietario y cola FIFO	Al ejecutar signal() la propiedad se transfiere directamente al primer proceso en espera, de modo que no se pierde ningún despertar.
Interbloqueo	Detección con grafo de	Con dos recursos de una sola instancia, cada

Tema	Decisión	Razón
	espera	proceso espera como máximo un recurso y cada recurso tiene un solo dueño; basta seguir la cadena de espera para encontrar un ciclo.
Arquitectura	Una clase Kernel como fachada	La CLI y la interfaz gráfica comparten exactamente la misma lógica, lo que evita duplicar reglas y errores.

Modelo de bloques. El siguiente diagrama muestra las dos interfaces, el Kernel que las atiende y los cuatro módulos que coordina.



1. Crear proceso: el Kernel llama a asignar(pid, tam); si hay hueco, el PCB entra a la cola de listos.
2. Proceso terminado: el planificador avisa al Kernel (alTerminar) y este llama a liberar(pid).
3. Sin hueco contiguo suficiente: error Out of Memory y el proceso se rechaza sin consumir PID.

Interacción entre el Planificador de Procesos y el Administrador de Memoria. Ambos módulos no se invocan entre sí de forma directa; el Kernel actúa como intermediario. La secuencia es la siguiente:

1. Al crear un proceso, el Kernel valida el tamaño (mayor que 0 MB) y la ráfaga (entre 1 y 1000) y solicita al administrador una partición con asignar(pid, tam).
2. Si existe un hueco contiguo suficiente, el administrador devuelve la dirección base y el Kernel crea el PCB, lo marca como LISTO y lo agrega a la cola del planificador.
3. Si no existe, el administrador lanza la excepción de desbordamiento, el contador de rechazados aumenta y el proceso no entra al sistema ni consume PID.

4. Cuando el planificador agota la ráfaga de un proceso, invoca la función de retorno alTerminar; el Kernel libera la RAM con liberar(pid), fusiona los huecos y libera también el semáforo y los recursos que el proceso retuviera.

Responsabilidad de cada clase.

Clase	Responsabilidad
PCB	Almacena PID, estado, tamaño en memoria, ráfaga total y restante, y datos auxiliares (base en RAM, llegada, finalización, bandera de aborto). Calcula tiempos de espera y de retorno.
Estado	Enumeración de los cinco estados: NUEVO, LISTO, EJECUCION, BLOQUEADO y TERMINADO.
PlanificadorRR	Cola de listos, quantum q = 3, cambio de contexto simulado, reloj, diagrama de Gantt.
AdministradorMemoria	RAM de 100 MB, lista de particiones, First-Fit, fusión de huecos, mapa de memoria.
DesbordamientoMemoriaException	Error Out of Memory lanzado cuando no hay hueco suficiente.
SemaforoBinario	Valor 1 o 0, propietario, cola de espera, operaciones wait() y signal().
Recurso y GestorRecursos	Dos recursos exclusivos con cola de espera; solicitud, liberación, detección de ciclo y liberación total de un proceso abortado.
Kernel	Integra los módulos, valida parámetros, crea y aborta procesos y genera los reportes.
Main	Interfaz CLI con menú de texto y lectura robusta de entradas.
MainGrafico, VentanaKernel, Graficos, Widgets, Tema	Interfaz gráfica Swing complementaria (KernelLab) sobre el mismo Kernel.
CasosPrueba, SimuladorCarrera, TeoriaRR	Casos de prueba automáticos, demostración de condición de carrera y calculadora teórica de Round Robin.

6.2 Planeación

a. Cronograma de actividades (WBS). La estructura de desglose del trabajo organiza el proyecto en cinco fases. Los periodos son de referencia y deben ajustarse a las fechas reales del curso.

Código	Actividad	Entregable	Periodo de referencia
1	**Inicio y análisis**		Semana 1
1.1	Lectura del enunciado y extracción de requerimientos	Lista de requerimientos	Semana 1
1.2	Selección de lenguaje y herramientas	Stack definido	Semana 1
2	**Diseño**		Semana 2
2.1	Modelo de bloques y reparto de clases	Diagrama de bloques	Semana 2
2.2	Diseño de estructuras (PCB, colas, mapa de memoria)	Especificación de clases	Semana 2
3	**Desarrollo**		Semanas 3 a 4
3.1	PCB y estados	Clases PCB y Estado	Semana 3
3.2	Planificador Round Robin	Clase PlanificadorRR	Semana 3
3.3	Administrador de memoria	Clase AdministradorMemoria	Semana 3
3.4	Semáforo binario y demostración de carrera	SemaforoBinario, SimuladorCarrera	Semana 4
3.5	Gestor de recursos y deadlock	GestorRecursos	Semana 4
3.6	Kernel integrador, validaciones y CLI	Kernel, Main	Semana 4
3.7	Interfaz gráfica complementaria	VentanaKernel y apoyo gráfico	Semana 4
4	**Pruebas**		Semana 5
4.1	Casos automáticos de la bitácora	CasosPrueba	Semana 5
4.2	Prueba de estrés y corrección de defectos	Registro de pruebas	Semana 5
5	**Documentación y entrega**		Semanas 6 a 7
5.1	Informe técnico en PDF	Este documento	Semana 6
5.2	Carga del código en GitHub	Repositorio público	Semana 6

Código	Actividad	Entregable	Periodo de referencia
5.3	Ensayo de la demostración en vivo	Presentación de 15 minutos	Semana 7

b. Roles de cada integrante (RACI). R = Responsable de ejecutar, A = quien Aprueba, C = Consultado, I = Informado. *Reemplazar cada rol por el nombre del integrante correspondiente. Si el proyecto fue individual, una sola persona asume todos los roles.*

Actividad	Líder de proyecto [Edgar García]	Desarrollo del núcleo [Angel Tzi]	Desarrollo de sincronización [Edgar y Angel]	Documentación y pruebas [Angel Morejón y Daliila Garcia]
Análisis de requerimientos	A / R	C	C	C
Diseño de arquitectura	A	R	C	I
Planificador RR y memoria	A	R	I	C
Semáforo y deadlock	A	C	R	C
Kernel integrador y CLI	A	R	R	I
Pruebas y estrés	A	C	C	R
Informe técnico	A	C	C	R
GitHub y demostración	A / R	R	R	C

c. Canal de comunicación.

Canal	Uso	Frecuencia
WhatsApp	Coordinación diaria y avisos	Diaria
GitHub (commits e issues)	Control de versiones, revisión de código y seguimiento de defectos	Continua
Reunión virtual o presencial	Seguimiento de avances y reparto de tareas	Semanal
Correo institucional	Consultas formales al catedrático	Según necesidad

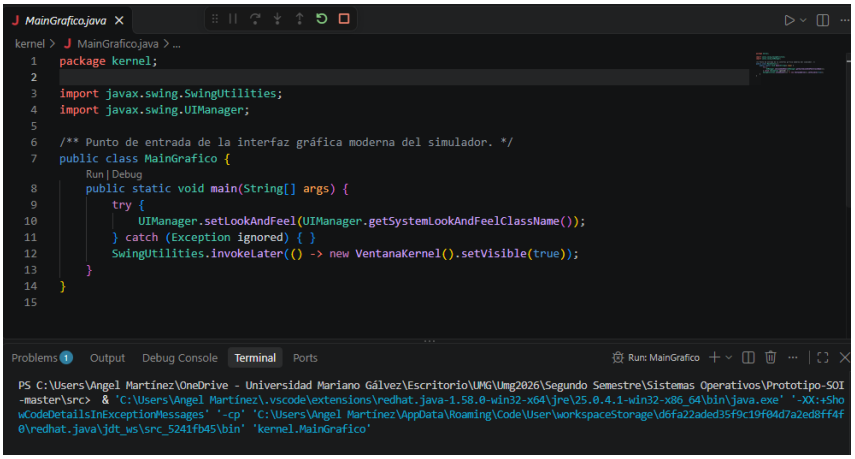
6.3 Desarrollo y ejecución

a. Stack tecnológico y versiones.

Componente	Detalle
Lenguaje	Java. El código usa únicamente la biblioteca estándar, por lo que es compatible con Java 8 o superior. Compilado y verificado con OpenJDK 21.0.10.
Interfaz de texto	Consola con menú numérico (clase Main)
Interfaz gráfica	Java Swing y AWT, incluidas en el JDK (clase MainGrafico)
Control de versiones	Git y GitHub
Dependencias externas	Ninguna

Variables de entorno. El simulador es autónomo: no utiliza conexiones a bases de datos, puertos de red, claves de API ni variables de entorno. Un único detalle de configuración es la codificación de caracteres: Main fuerza la salida en UTF-8 para mostrar correctamente tildes y símbolos.

Compilación y ejecución. Desde la carpeta raíz del proyecto:



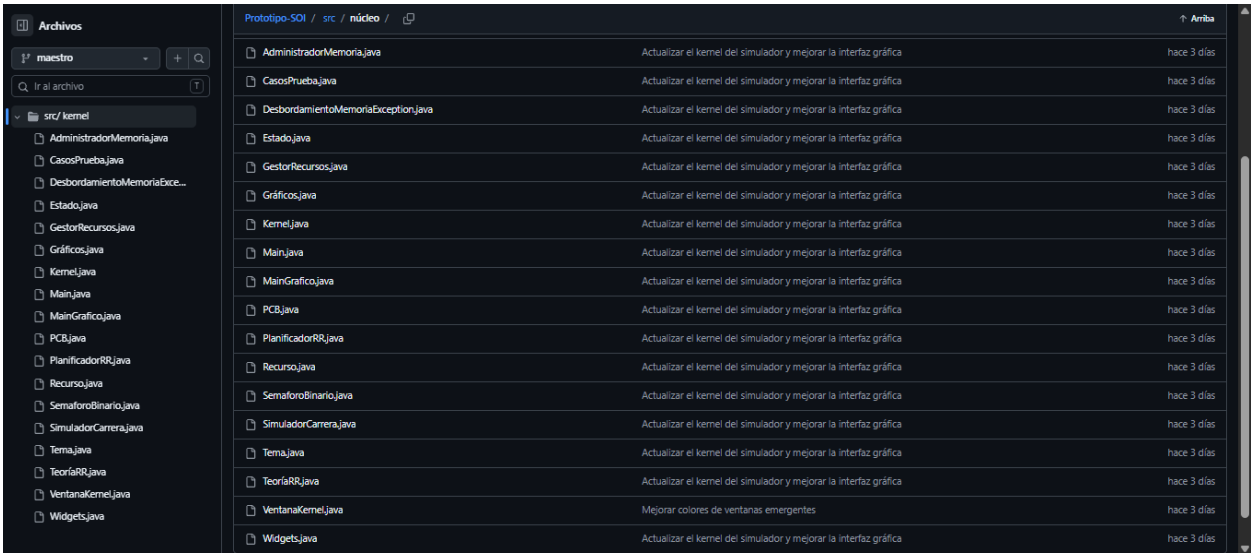
b. Requerimientos específicos.

Código	Requerimiento del enunciado	Cumplimiento en el prototipo
RF-01	PCB con PID, estado, tamaño, ráfaga total y restante	Clase PCB con esos cinco campos
RF-02	Round Robin con quantum q = 3 y cambio de contexto simulado	PlanificadorRR: constante QUANTUM = 3; si el proceso no termina, regresa al final de la cola y se cuenta un cambio de contexto

Código	Requerimiento del enunciado	Cumplimiento en el prototipo
RF-03	RAM de 100 MB y rechazo con error de desbordamiento	AdministradorMemoria con TOTAL_MB = 100 y DesbordamientoMemoriaException
RF-04	Semáforo binario con wait() y signal() y cola de espera	SemaforoBinario con propietario y cola FIFO
RF-05	Dos recursos exclusivos y espera circular forzada	GestorRecursos y Kernel.forzarDeadlock(A, B)
RF-06	Alerta de deadlock y comando para abortar un proceso	Kernel.detectarDeadlock() y Kernel.abortarProceso(pid), opciones 15 y 16 del menú
RF-07	Interfaz interactiva por menús de texto	Main, 19 opciones más salir
RF-08	Robustez ante parámetros aleatorios	Validación en Kernel y Main; opción 4 genera procesos aleatorios, incluidos valores inválidos
RNF-01	Código modular, limpio y comentado	Un archivo por responsabilidad, con comentarios de documentación en cada clase
RNF-02	Código cargado en GitHub	Repositorio: [URL del repositorio]

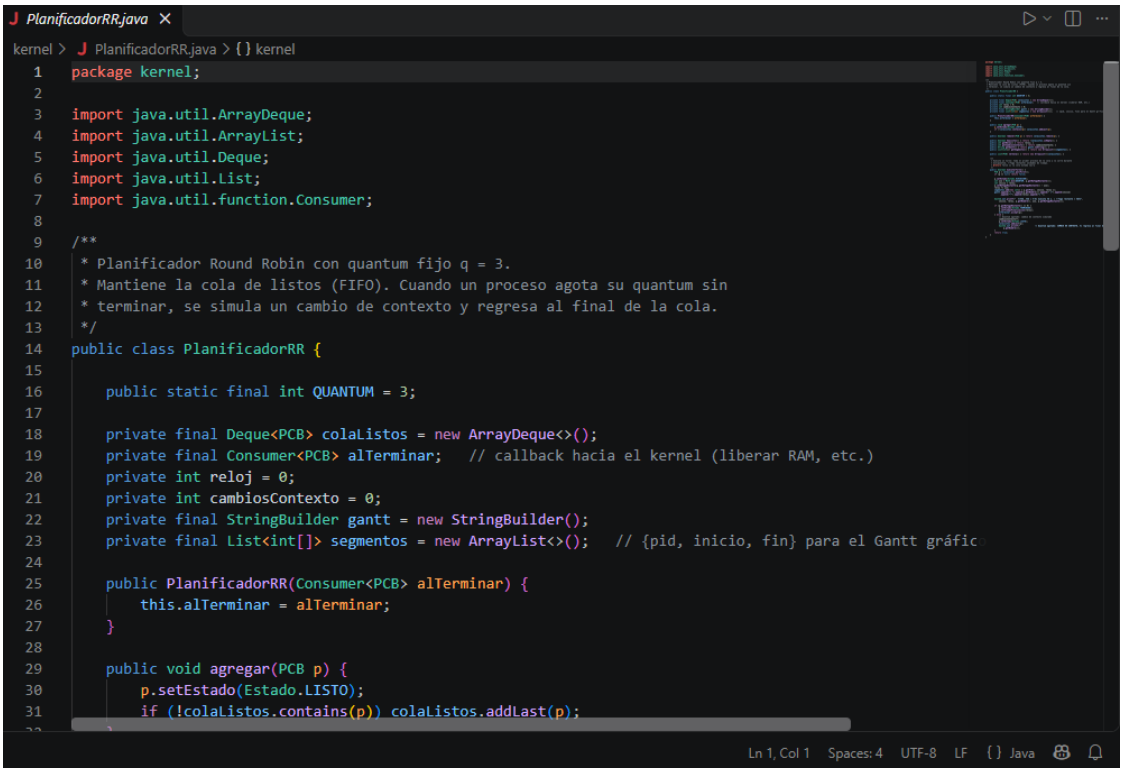
c. Construcción del prototipo.

Estructura del repositorio.



Planificador Round Robin. Cada turno toma el primer proceso de la cola, lo pone en EJECUCION y le concede el menor valor entre el quantum y su ráfaga restante. Avanza el reloj, registra el segmento para el diagrama de Gantt y, si la ráfaga llegó a cero, marca el proceso como TERMINADO y avisa al Kernel; en

caso contrario incrementa el contador de cambios de contexto y lo reencola al final.



```java
package kernel;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;
import java.util.function.Consumer;

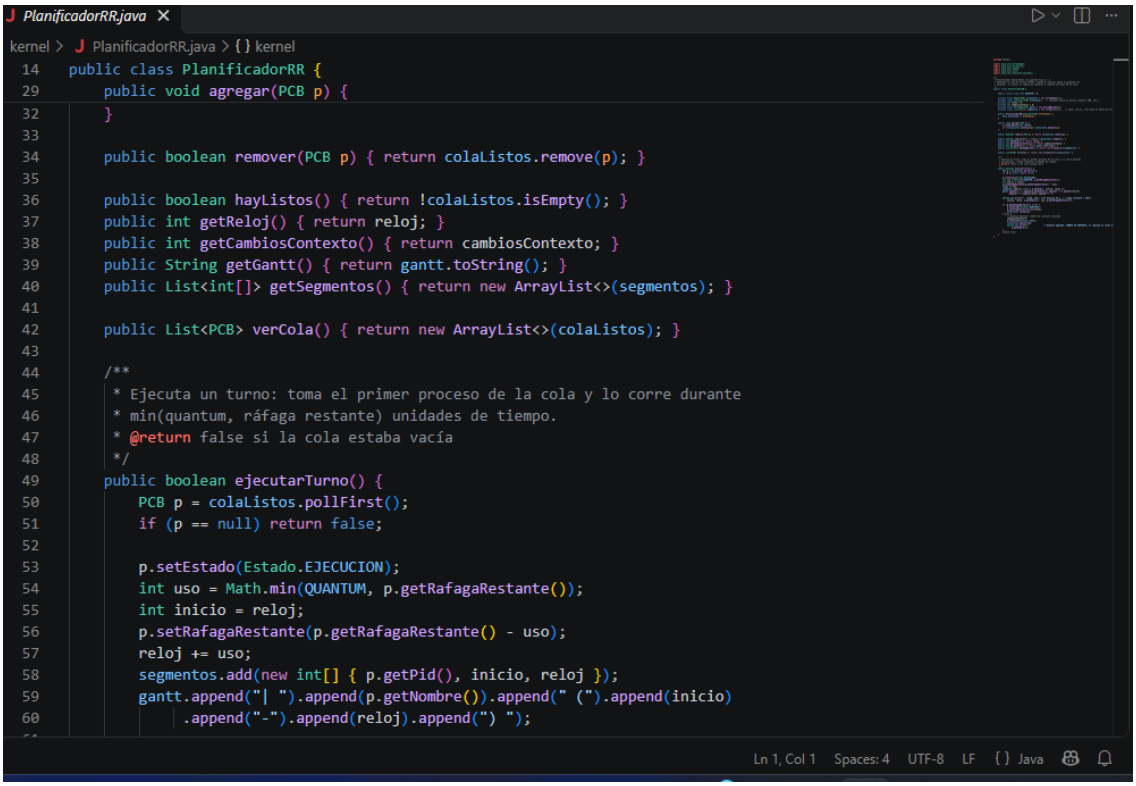
/**
 * Planificador Round Robin con quantum fijo q = 3.
 * Mantiene la cola de listos (FIFO). Cuando un proceso agota su quantum sin
 * terminar, se simula un cambio de contexto y regresa al final de la cola.
 */
public class PlanificadorRR {

    public static final int QUANTUM = 3;

    private final Deque<PCB> colaListos = new ArrayDeque<>();
    private final Consumer<PCB> alTerminar;   // callback hacia el kernel (liberar RAM, etc.)
    private int reloj = 0;
    private int cambiosContexto = 0;
    private final StringBuilder gantt = new StringBuilder();
    private final List<int[]> segmentos = new ArrayList<>();   // {pid, inicio, fin} para el Gantt gráfic

    public PlanificadorRR(Consumer<PCB> alTerminar) {
        this.alTerminar = alTerminar;
    }

    public void agregar(PCB p) {
        p.setEstado(Estado.LISTO);
        if (!colaListos.contains(p)) colaListos.addLast(p);
```



```java
public class PlanificadorRR {
    public void agregar(PCB p) {
    }

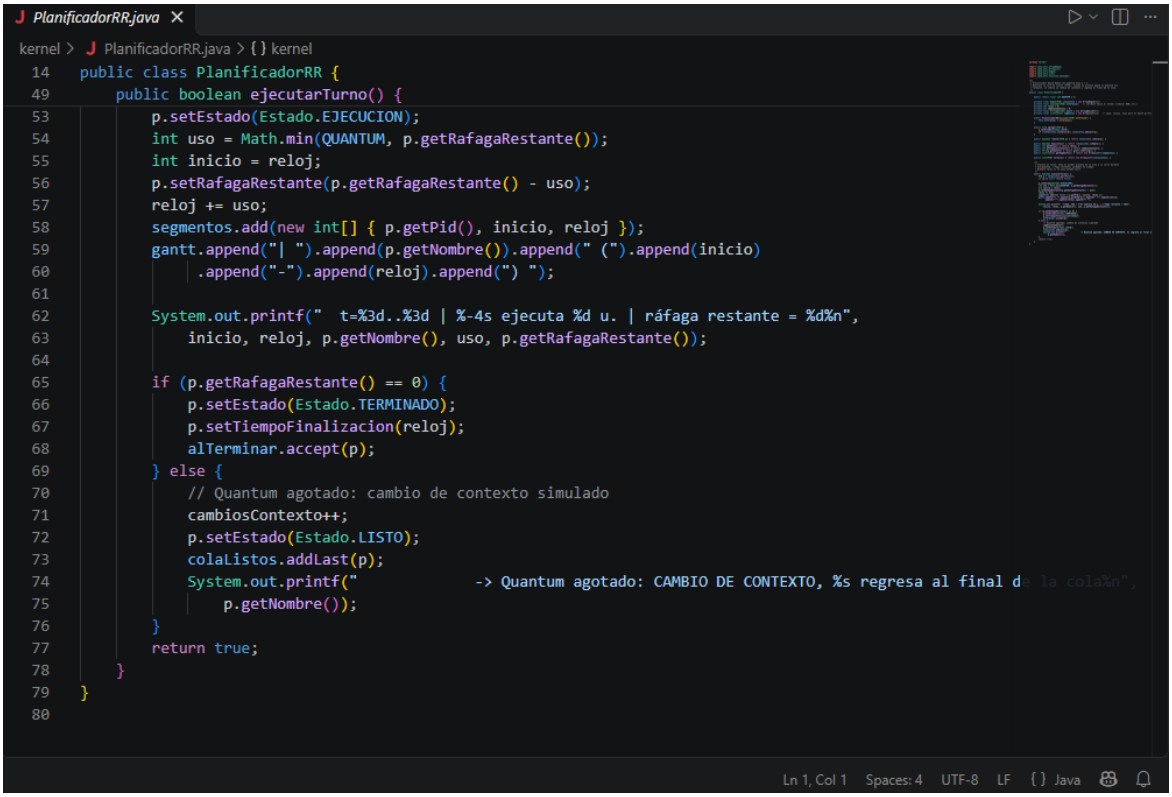
    public boolean remover(PCB p) { return colaListos.remove(p); }

    public boolean hayListos() { return !colaListos.isEmpty(); }
    public int getReloj() { return reloj; }
    public int getCambiosContexto() { return cambiosContexto; }
    public String getGantt() { return gantt.toString(); }
    public List<int[]> getSegmentos() { return new ArrayList<>(segmentos); }

    public List<PCB> verCola() { return new ArrayList<>(colaListos); }

    /**
     * Ejecuta un turno: toma el primer proceso de la cola y lo corre durante
     * min(quantum, ráfaga restante) unidades de tiempo.
     * @return false si la cola estaba vacía
     */
    public boolean ejecutarTurno() {
        PCB p = colaListos.pollFirst();
        if (p == null) return false;

        p.setEstado(Estado.EJECUCION);
        int uso = Math.min(QUANTUM, p.getRafagaRestante());
        int inicio = reloj;
        p.setRafagaRestante(p.getRafagaRestante() - uso);
        reloj += uso;
        segmentos.add(new int[] { p.getPid(), inicio, reloj });
        gantt.append("| ").append(p.getNombre()).append(" (").append(inicio)
             .append("-").append(reloj).append(") ");
```



Administrator de memoria. Mantiene una lista de particiones, cada una con base, tamaño y propietario (un PID, o -1 si es hueco). Para asignar recorre la lista y usa el primer hueco que alcance (First-Fit); si el hueco es mayor que lo pedido, lo divide en la partición del proceso y un hueco restante. Al liberar, marca el espacio como libre y fusiona los huecos adyacentes. El mensaje de error informa la memoria solicitada, la libre en total y el mayor hueco contiguo, lo que permite distinguir un desbordamiento por falta de espacio de uno por fragmentación.

AdministradorMemoria.java

kernel > AdministradorMemoria.java > { } kernel

```java
package kernel;

import java.util.ArrayList;
import java.util.List;

/**
 * Administrador de Memoria: RAM física simulada de 100 MB.
 * Estrategia: Asignación Contigua Dinámica (particiones variables) con
 * política First-Fit. Al liberar, los huecos adyacentes se fusionan.
 * Si no existe un hueco contiguo suficiente se lanza Out of Memory.
 */
public class AdministradorMemoria {

    public static final int TOTAL_MB = 100;

    /** Partición de memoria. pid = -1 significa hueco libre. */
    private static class Bloque {
        int base;
        int tam;
        int pid;
        Bloque(int base, int tam, int pid) {
            this.base = base;
            this.tam = tam;
            this.pid = pid;
        }
    }

    private final List<Bloque> bloques = new ArrayList<>();

    public AdministradorMemoria() {
        bloques.add(new Bloque(base: 0, TOTAL_MB, -1)); // toda la RAM libre
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Java

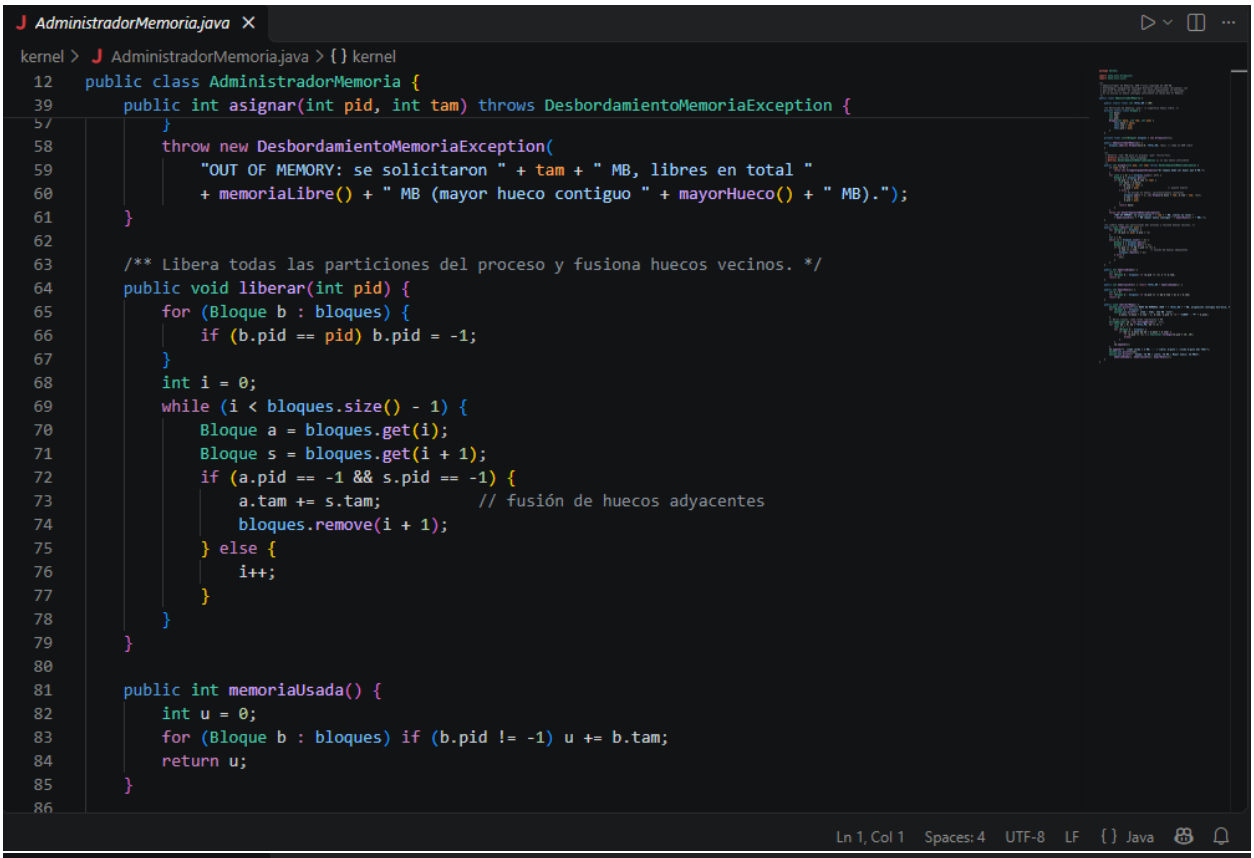
AdministradorMemoria.java

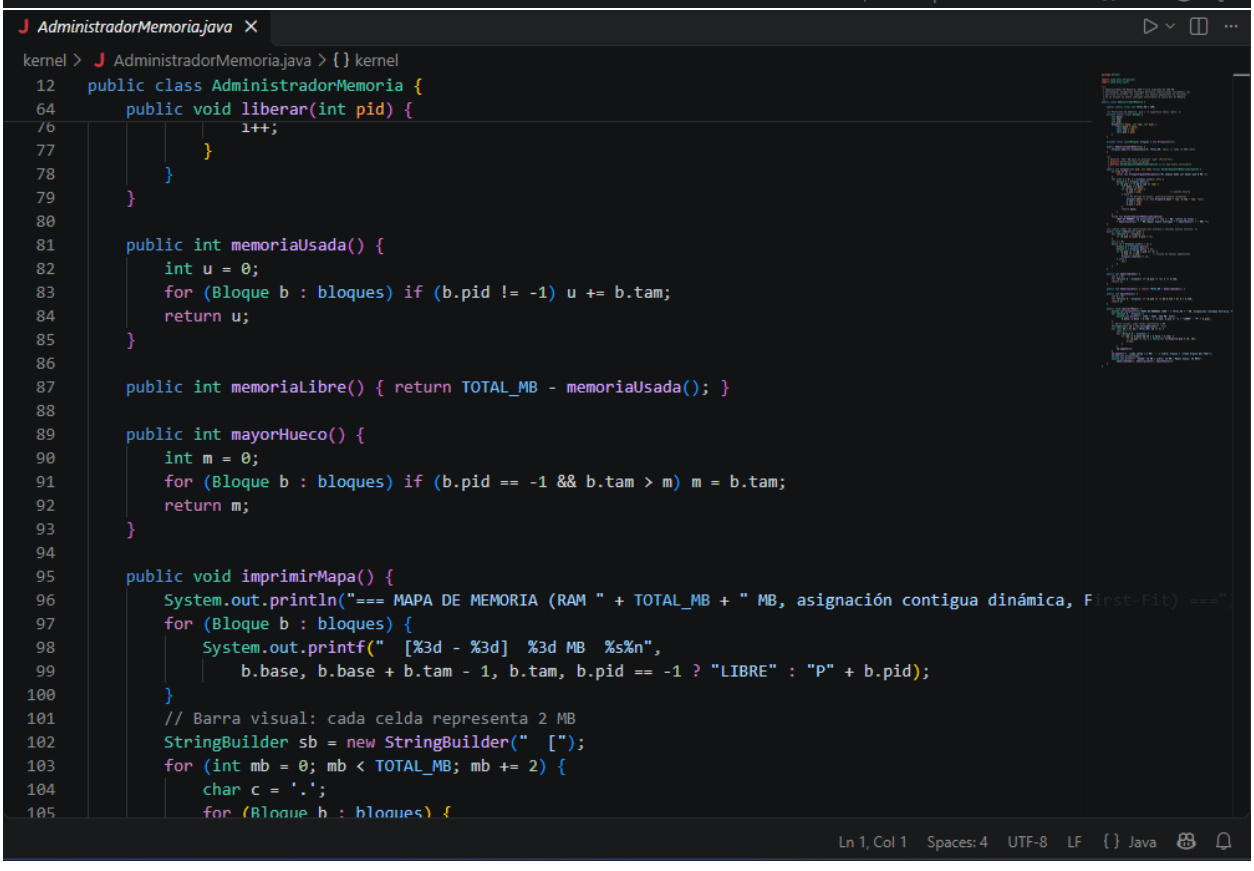
kernel > AdministradorMemoria.java > { } kernel

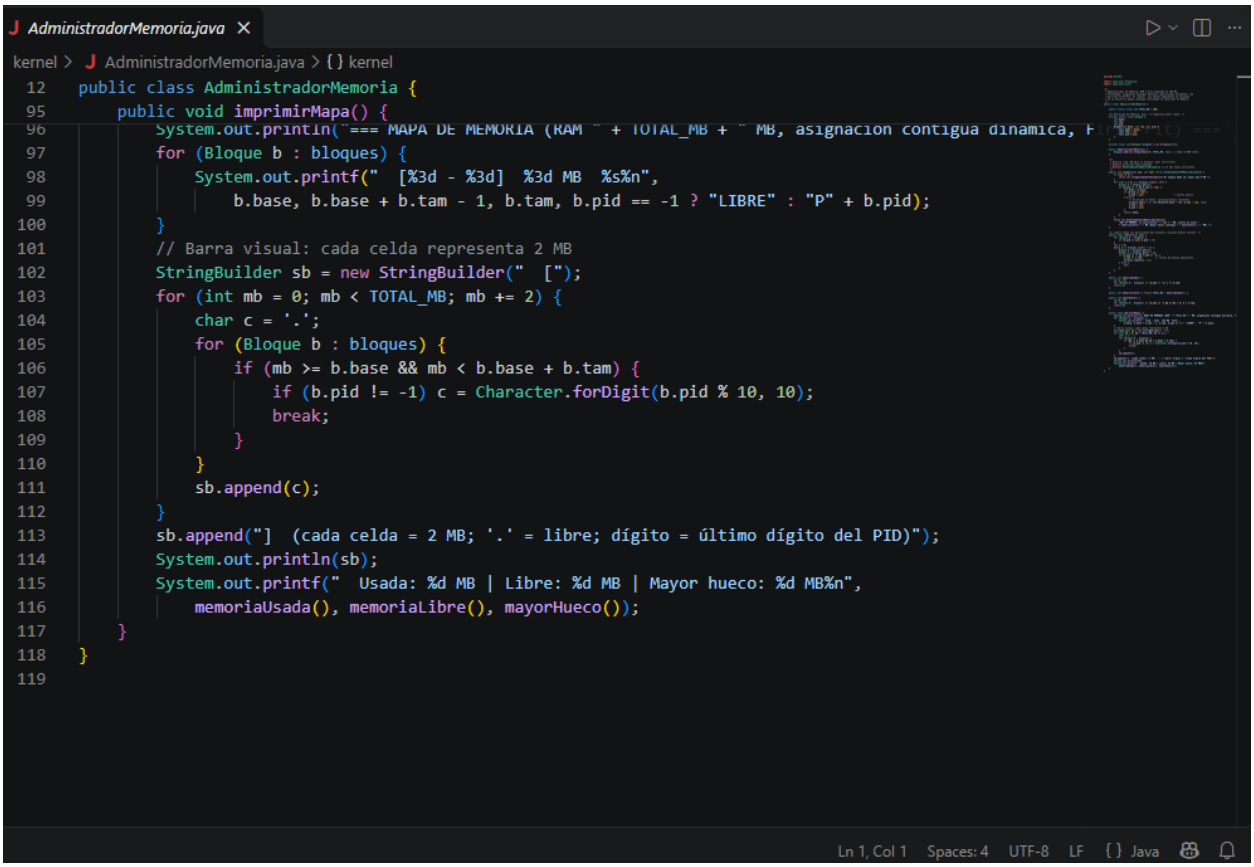
```java
public class AdministradorMemoria {
    public AdministradorMemoria() {
        bloques.add(new Bloque(base: 0, TOTAL_MB, -1)); // toda la RAM libre
    }

    /**
     * Reserva 'tam' MB para el proceso 'pid' (First-Fit).
     * @return dirección base asignada
     * @throws DesbordamientoMemoriaException si no hay hueco suficiente
     */
    public int asignar(int pid, int tam) throws DesbordamientoMemoriaException {
        if (tam <= 0) {
            throw new IllegalArgumentException("El tamaño debe ser mayor que 0 MB.");
        }
        for (int i = 0; i < bloques.size(); i++) {
            Bloque b = bloques.get(i);
            if (b.pid == -1 && b.tam >= tam) {
                int base = b.base;
                if (b.tam == tam) {
                    b.pid = pid;                    // ajuste exacto
                } else {
                    // Se divide el hueco: [proceso][hueco restante]
                    bloques.add(i + 1, new Bloque(b.base + tam, b.tam - tam, -1));
                    b.tam = tam;
                    b.pid = pid;
                }
                return base;
            }
        }
        throw new DesbordamientoMemoriaException(
            "OUT OF MEMORY: se solicitaron " + tam + " MB, libres en total "
            + memoriaLibre() + " MB (mayor hueco contiguo " + mayorHueco() + " MB).");
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Java


AdministradorMemoria.java
kernel > AdministradorMemoria.java > { } kernel
12 public class AdministradorMemoria {
39 public int asignar(int pid, int tam) throws DesbordamientoMemoriaException {
57 }
58 throw new DesbordamientoMemoriaException(
59 "OUT OF MEMORY: se solicitaron " + tam + " MB, libres en total "
60 + memoriaLibre() + " MB (mayor hueco contiguo " + mayorHueco() + " MB).");
61 }
62
63 /** Libera todas las particiones del proceso y fusiona huecos vecinos. */
64 public void liberar(int pid) {
65 for (Bloque b : bloques) {
66 if (b.pid == pid) b.pid = -1;
67 }
68 int i = 0;
69 while (i < bloques.size() - 1) {
70 Bloque a = bloques.get(i);
71 Bloque s = bloques.get(i + 1);
72 if (a.pid == -1 && s.pid == -1) {
73 a.tam += s.tam; // fusión de huecos adyacentes
74 bloques.remove(i + 1);
75 } else {
76 i++;
77 }
78 }
79 }
80
81 public int memoriaUsada() {
82 int u = 0;
83 for (Bloque b : bloques) if (b.pid != -1) u += b.tam;
84 return u;
85 }
86
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Java


AdministradorMemoria.java
kernel > AdministradorMemoria.java > { } kernel
12 public class AdministradorMemoria {
64 public void liberar(int pid) {
76 i++;
77 }
78 }
79 }
80
81 public int memoriaUsada() {
82 int u = 0;
83 for (Bloque b : bloques) if (b.pid != -1) u += b.tam;
84 return u;
85 }
86
87 public int memoriaLibre() { return TOTAL_MB - memoriaUsada(); }
88
89 public int mayorHueco() {
90 int m = 0;
91 for (Bloque b : bloques) if (b.pid == -1 && b.tam > m) m = b.tam;
92 return m;
93 }
94
95 public void imprimirMapa() {
96 System.out.println("=== MAPA DE MEMORIA (RAM " + TOTAL_MB + " MB, asignación contigua dinámica, First-Fit) ===");
97 for (Bloque b : bloques) {
98 System.out.printf(" [%3d - %3d] %3d MB %s%n",
99 b.base, b.base + b.tam - 1, b.tam, b.pid == -1 ? "LIBRE" : "P" + b.pid);
100 }
101 // Barra visual: cada celda representa 2 MB
102 StringBuilder sb = new StringBuilder(" [");
103 for (int mb = 0; mb < TOTAL_MB; mb += 2) {
104 char c = '.';
105 for (Bloque b : bloques) {
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Java



Semáforo binario. wait() entrega la sección crítica si el valor es 1; si no, el proceso pasa a BLOQUEADO y a la cola de espera. signal() solo puede ser invocado por el propietario; si hay procesos esperando, transfiere la propiedad al primero y lo pasa a LISTO, y si no, devuelve el valor a 1.

SemaforoBinario.java

kernel > SemaforoBinario.java > { } kernel

```java

import java.util.LinkedList;
import java.util.Queue;

/**
 * Semáforo binario (valor 1 = libre, 0 = ocupado) con cola de espera propia.
 * Protege la sección crítica (variable global compartida).
 *  - wait(): si está ocupado, el proceso pasa a BLOQUEADO y a la cola de espera.
 *  - signal(): si hay procesos esperando, se despierta al primero (FIFO) y
 *              se le transfiere la propiedad; si no, el semáforo queda libre.
 */
public class SemaforoBinario {
    private int valor = 1;
    private PCB propietario = null;
    private final Queue<PCB> colaEspera = new LinkedList<>();

    /** @return true si el proceso entró a la sección crítica; false si quedó bloqueado. */
    public boolean wait(PCB p) {
        if (propietario == p) {
            throw new IllegalStateException(p.getNombre() + " ya se encuentra dentro de la sección crític
        }
        if (colaEspera.contains(p)) {
            throw new IllegalStateException(p.getNombre() + " ya está en la cola de espera del semáforo.");
        }
        if (valor == 1) {
            valor = 0;
            propietario = p;
            return true;
        }
        colaEspera.add(p);
        p.setEstado(Estado.BLOQUEADO);
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Java

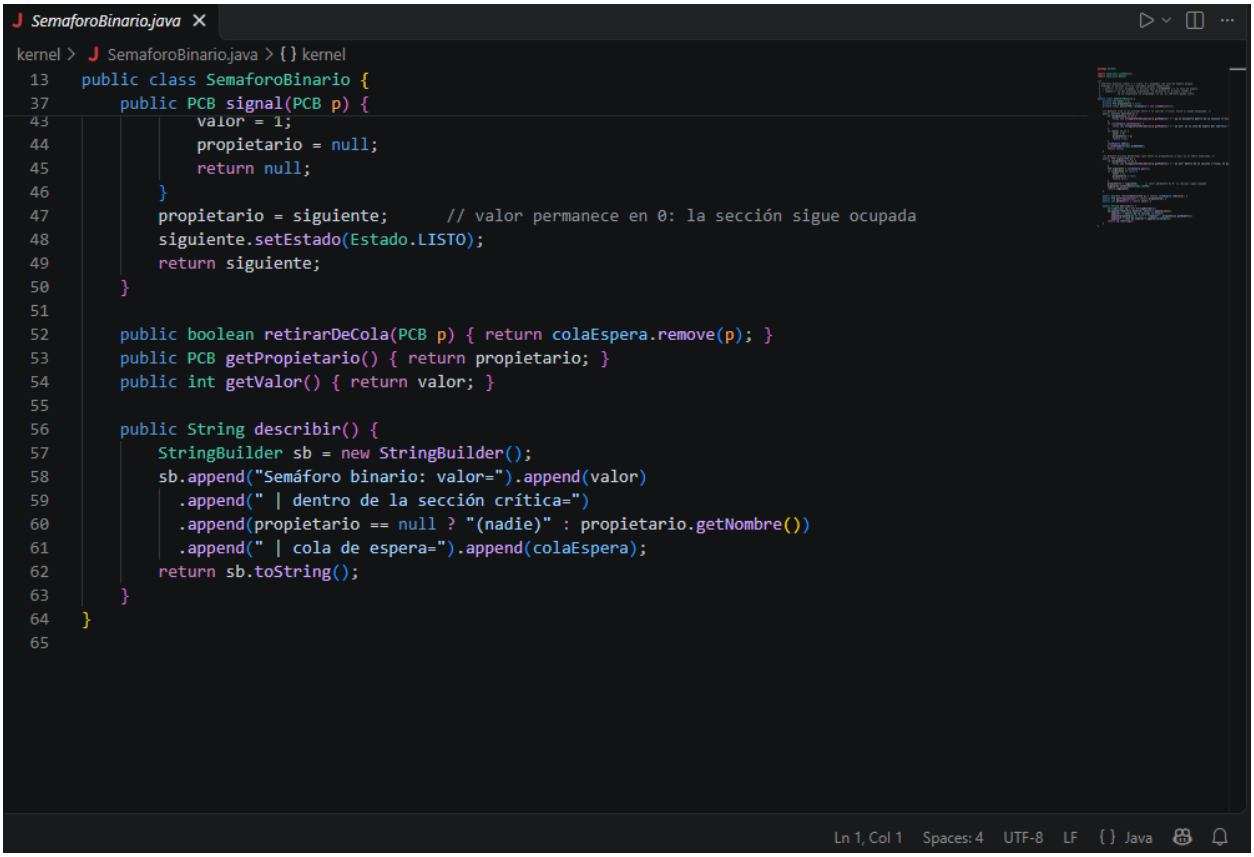
SemaforoBinario.java

kernel > SemaforoBinario.java > { } kernel

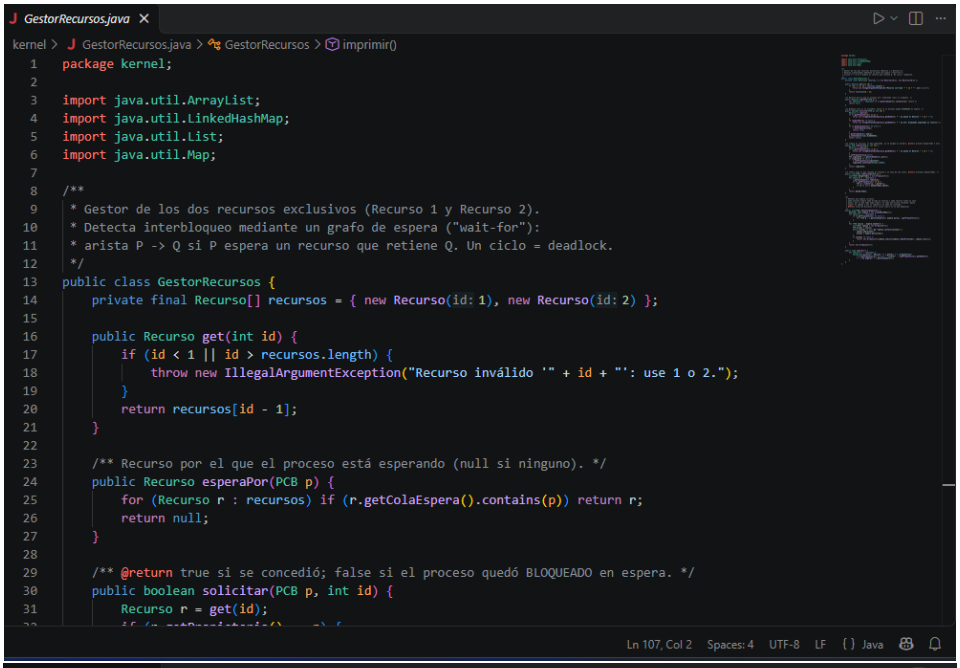
```java
public class SemaforoBinario {
    public boolean wait(PCB p) {
            throw new IllegalStateException(p.getNombre() + " ya se encuentra dentro de la seccion critic
        }
        if (colaEspera.contains(p)) {
            throw new IllegalStateException(p.getNombre() + " ya está en la cola de espera del semáforo.");
        }
        if (valor == 1) {
            valor = 0;
            propietario = p;
            return true;
        }
        colaEspera.add(p);
        p.setEstado(Estado.BLOQUEADO);
        return false;
    }

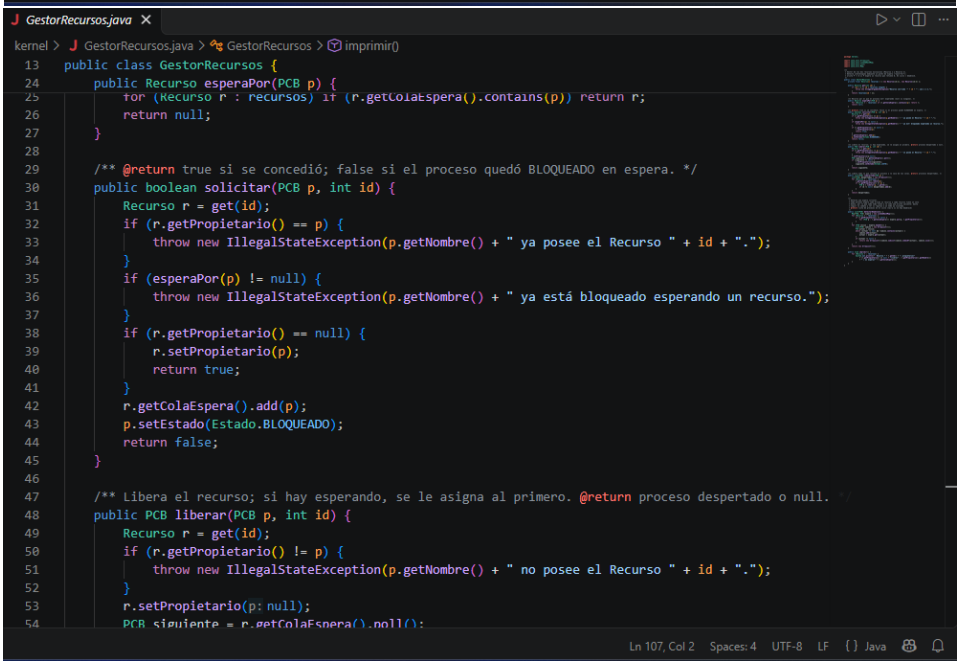
    /** @return proceso despertado (que ahora es propietario) o null si no había esperando. */
    public PCB signal(PCB p) {
        if (propietario != p) {
            throw new IllegalStateException(p.getNombre() + " no está dentro de la sección crítica; no pu
        }
        PCB siguiente = colaEspera.poll();
        if (siguiente == null) {
            valor = 1;
            propietario = null;
            return null;
        }
        propietario = siguiente;      // valor permanece en 0: la sección sigue ocupada
        siguiente.setEstado(Estado.LISTO);
        return siguiente;
    }
```

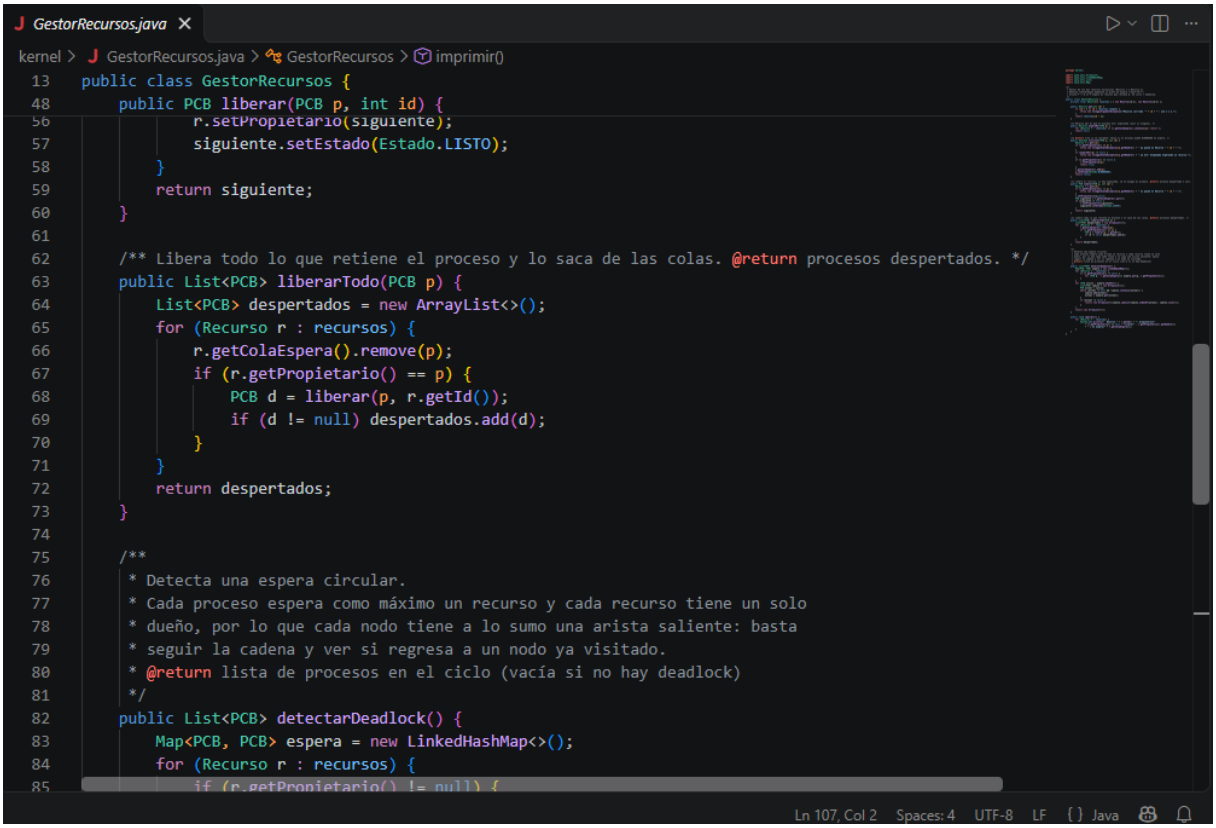
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Java


SemaforoBinario.java
kernel > SemaforoBinario.java > { } kernel
13 public class SemaforoBinario {
37 public PCB signal(PCB p) {
43 valor = 1;
44 propietario = null;
45 return null;
46 }
47 propietario = siguiente; // valor permanece en 0: la sección sigue ocupada
48 siguiente.setEstado(Estado.LISTO);
49 return siguiente;
50 }
51
52 public boolean retirarDeCola(PCB p) { return colaEspera.remove(p); }
53 public PCB getPropietario() { return propietario; }
54 public int getValor() { return valor; }
55
56 public String describir() {
57 StringBuilder sb = new StringBuilder();
58 sb.append("Semáforo binario: valor=").append(valor)
59 .append(" | dentro de la sección crítica=")
60 .append(propietario == null ? "(nadie)" : propietario.getNombre())
61 .append(" | cola de espera=").append(colaEspera);
62 return sb.toString();
63 }
64 }
65
Ln 1, Col 1 Spaces: 4 UTF-8 LF { } Java

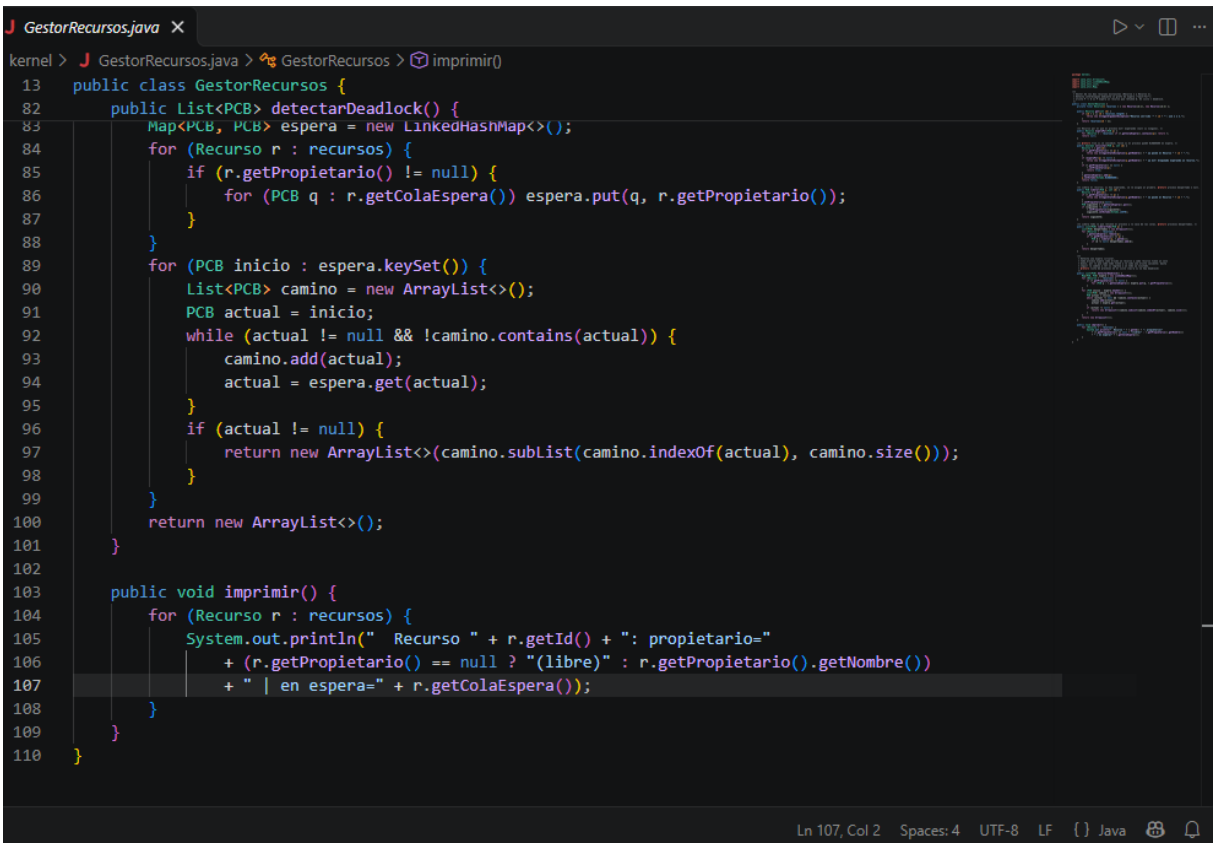
Gestor de recursos y deadlock. Cada recurso tiene un propietario y una cola. El método de detección construye el grafo de espera (cada proceso bloqueado apunta al dueño del recurso que espera) y sigue la cadena desde cada nodo; si regresa a un nodo ya visitado, el tramo repetido es el ciclo de interbloqueo. Para resolverlo, abortarProceso(pid) retira al proceso de las colas, libera su RAM, el semáforo y los recursos, y despierta a quien esperaba por ellos.







```
GestorRecursos.java
kernel > GestorRecursos.java > GestorRecursos > imprimir()
13  public class GestorRecursos {
48      public PCB liberar(PCB p, int id) {
56              r.setPropietario(siguiente);
57              siguiente.setEstado(Estado.LISTO);
58          }
59          return siguiente;
60      }
61
62      /** Libera todo lo que retiene el proceso y lo saca de las colas. @return procesos despertados. */
63      public List<PCB> liberarTodo(PCB p) {
64          List<PCB> despertados = new ArrayList<>();
65          for (Recurso r : recursos) {
66              r.getColaEspera().remove(p);
67              if (r.getPropietario() == p) {
68                  PCB d = liberar(p, r.getId());
69                  if (d != null) despertados.add(d);
70              }
71          }
72          return despertados;
73      }
74
75      /**
76       * Detecta una espera circular.
77       * Cada proceso espera como máximo un recurso y cada recurso tiene un solo
78       * dueño, por lo que cada nodo tiene a lo sumo una arista saliente: basta
79       * seguir la cadena y ver si regresa a un nodo ya visitado.
80       * @return lista de procesos en el ciclo (vacía si no hay deadlock)
81       */
82      public List<PCB> detectarDeadlock() {
83          Map<PCB, PCB> espera = new LinkedHashMap<>();
84          for (Recurso r : recursos) {
85              if (r.getPropietario() != null) {
Ln 107, Col 2  Spaces: 4  UTF-8  LF  { } Java
```

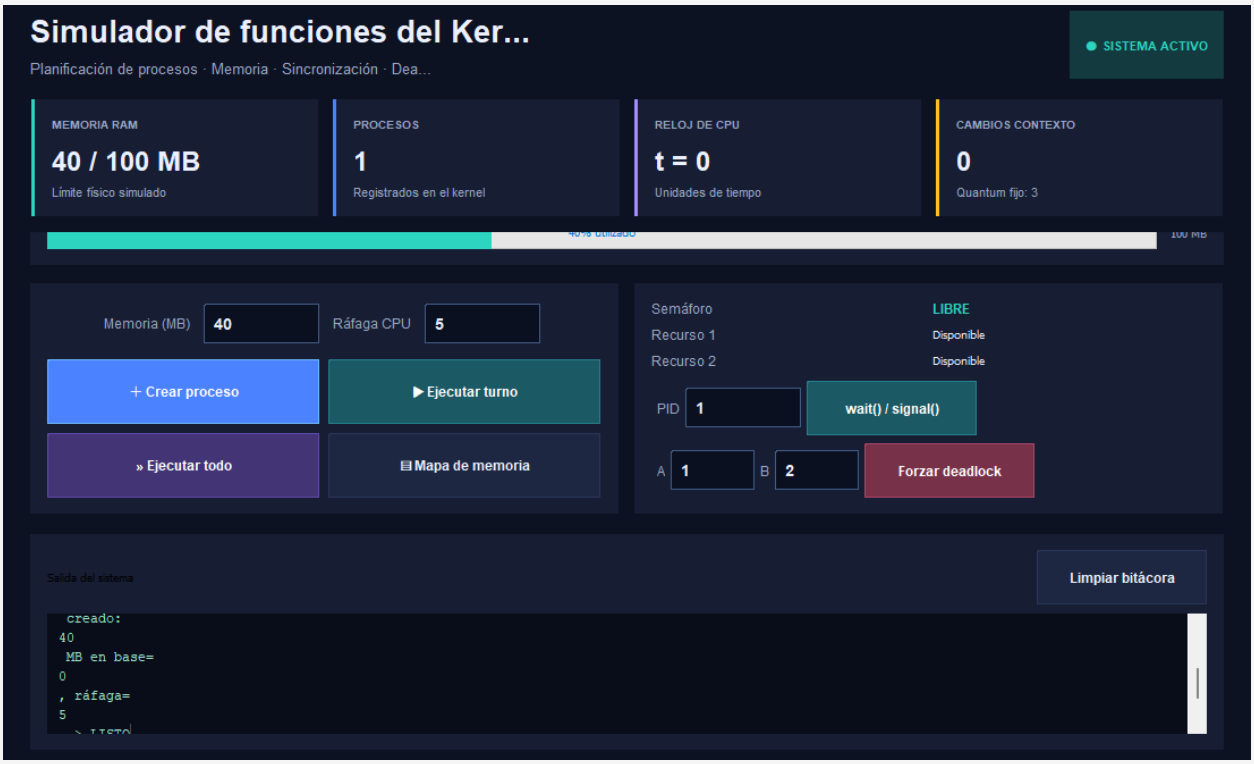


```
GestorRecursos.java
kernel > GestorRecursos.java > GestorRecursos > imprimir()
13  public class GestorRecursos {
82      public List<PCB> detectarDeadlock() {
83          Map<PCB, PCB> espera = new LinkedHashMap<>();
84          for (Recurso r : recursos) {
85              if (r.getPropietario() != null) {
86                  for (PCB q : r.getColaEspera()) espera.put(q, r.getPropietario());
87              }
88          }
89          for (PCB inicio : espera.keySet()) {
90              List<PCB> camino = new ArrayList<>();
91              PCB actual = inicio;
92              while (actual != null && !camino.contains(actual)) {
93                  camino.add(actual);
94                  actual = espera.get(actual);
95              }
96              if (actual != null) {
97                  return new ArrayList<>(camino.subList(camino.indexOf(actual), camino.size()));
98              }
99          }
100         return new ArrayList<>();
101     }
102
103     public void imprimir() {
104         for (Recurso r : recursos) {
105             System.out.println("  Recurso " + r.getId() + ": propietario="
106                 + (r.getPropietario() == null ? "(libre)" : r.getPropietario().getNombre())
107                 + " | en espera=" + r.getColaEspera());
108         }
109     }
110 }
Ln 107, Col 2  Spaces: 4  UTF-8  LF  { } Java
```

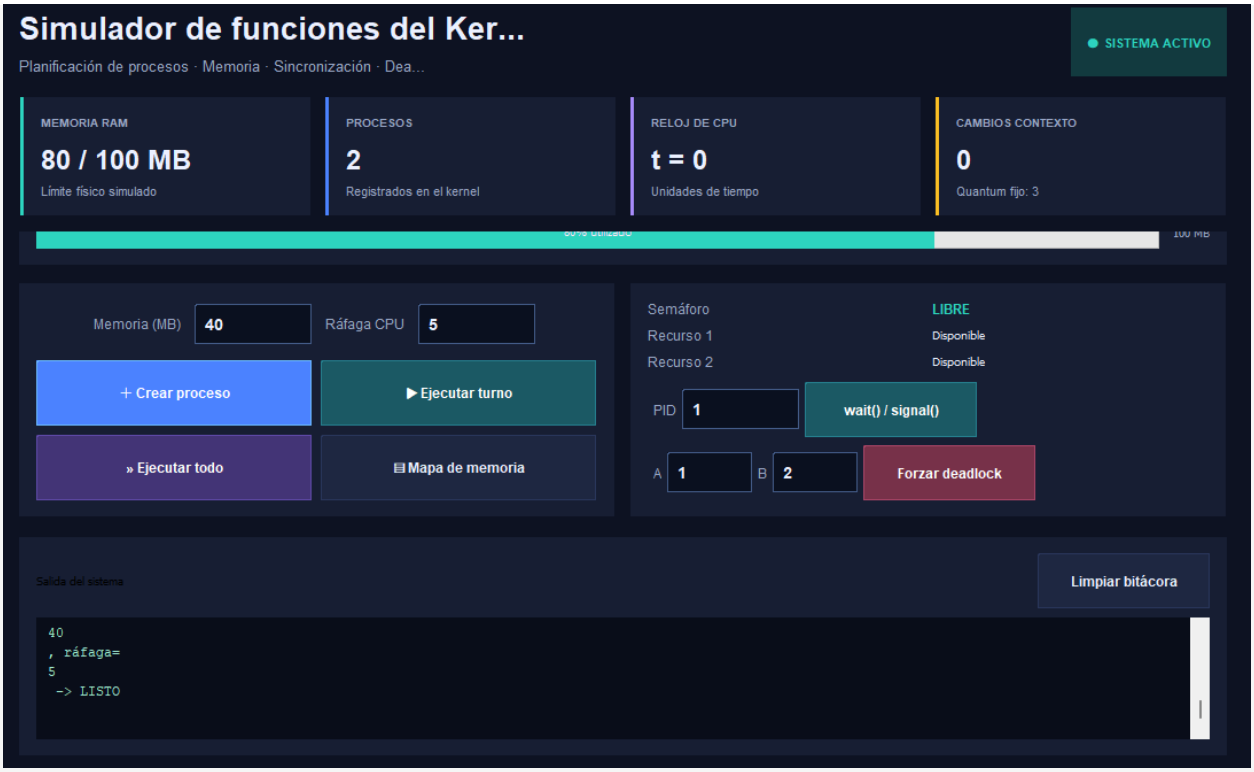
d. Pruebas del prototipo (bitácora de casos de prueba). Los cuatro casos se ejecutan desde la opción 18 del menú (Casos de prueba automáticos). Cada caso usa un Kernel nuevo e independiente, por lo que es reproducible. A continuación se presentan volcados de texto reales de la ejecución, comentados.

Caso 1. Rechazo de procesos por falta de RAM. Se solicitan procesos de 40, 40, 30, 20 y 1 MB, más dos tamaños inválidos (0 y -5).

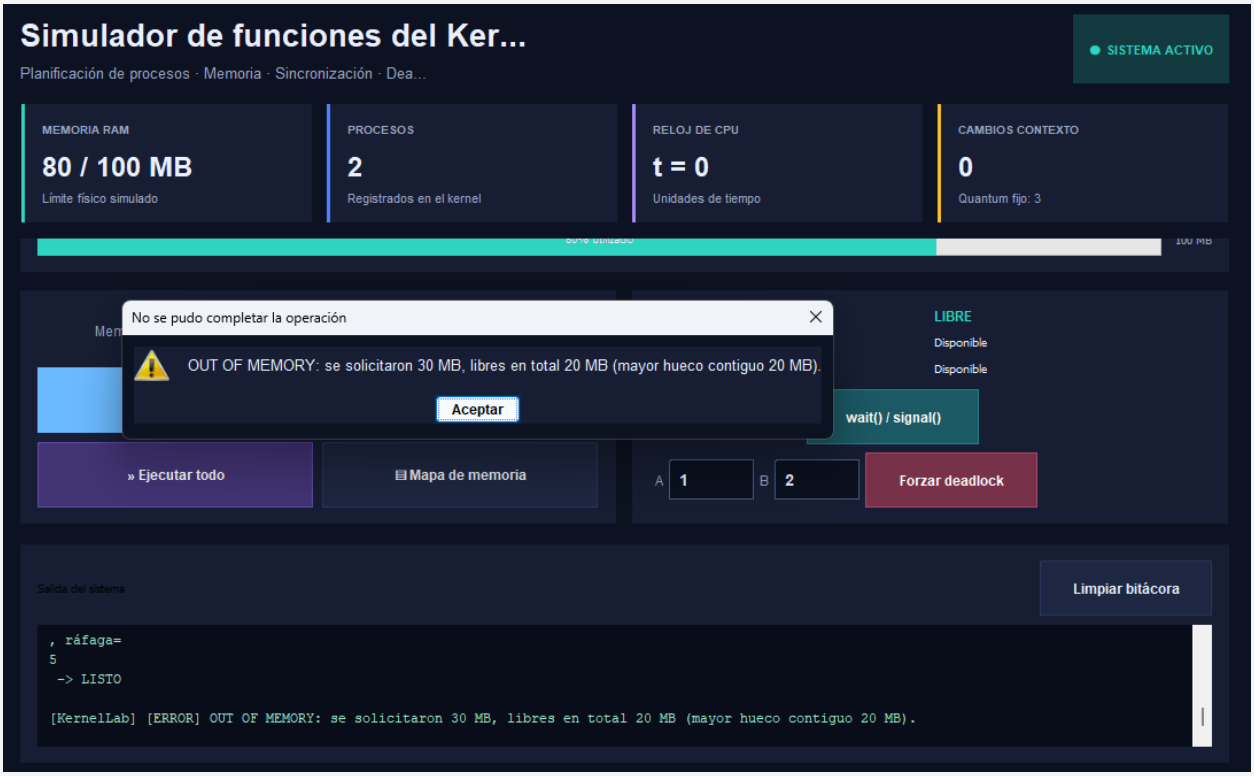
```
> crearProceso(tam=40 MB, ráfaga=5)
  [OK] Proceso P1 creado: 40 MB en base=0, ráfaga=5 -> LISTO
```



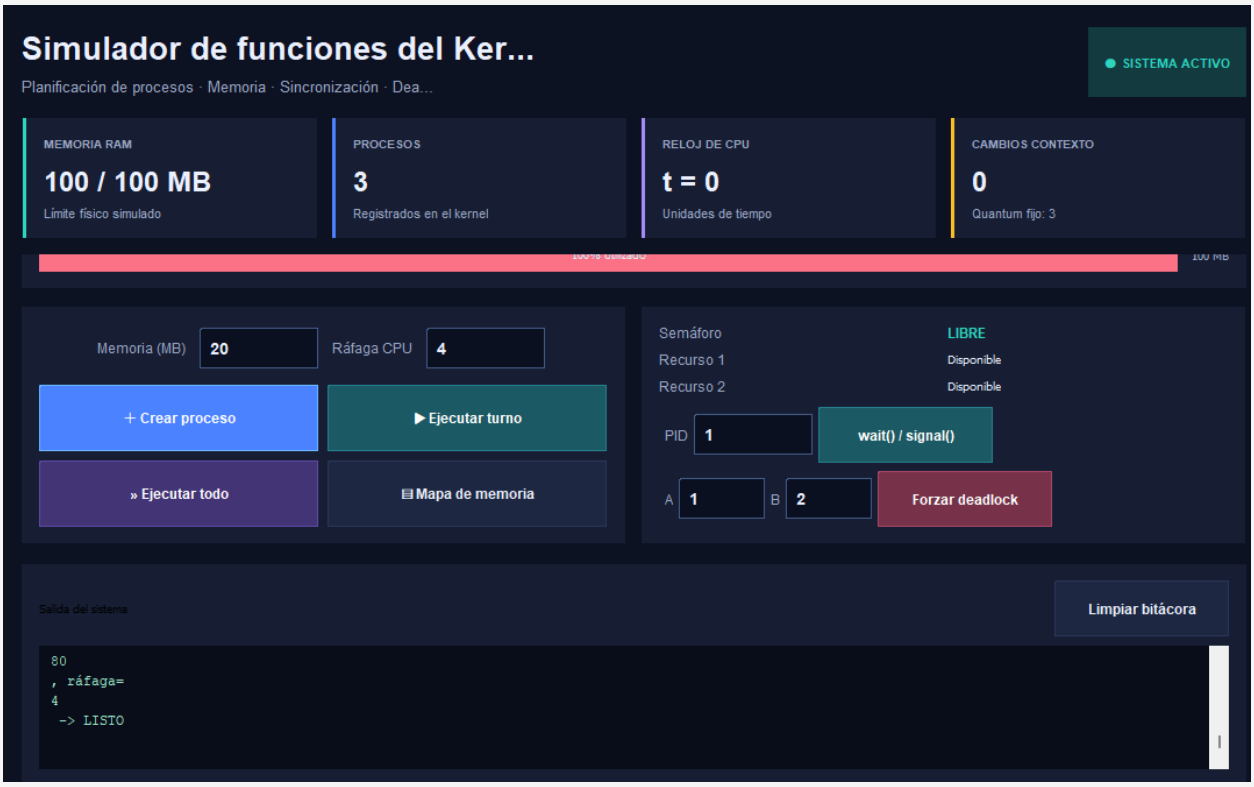

```
> crearProceso(tam=40 MB, ráfaga=5)
  [OK] Proceso P2 creado: 40 MB en base=40, ráfaga=5 -> LISTO
```



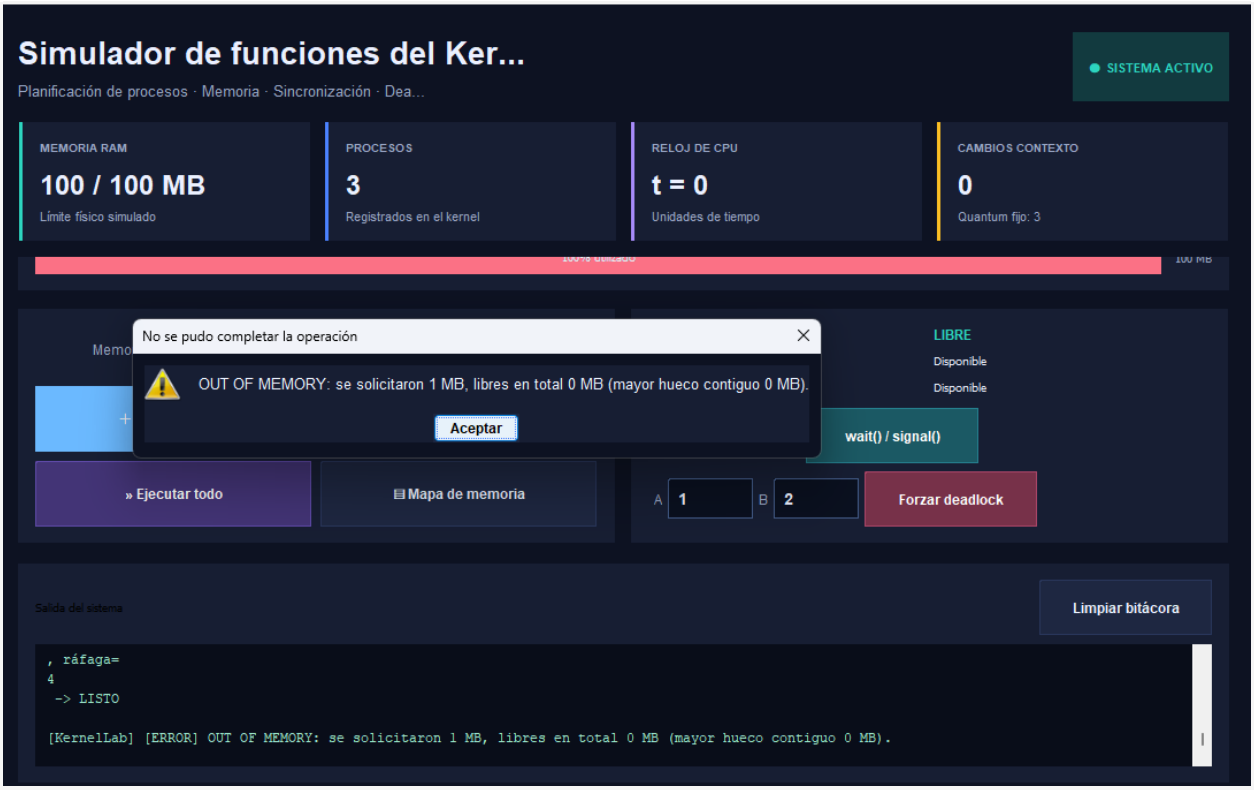

```
> crearProceso(tam=30 MB, ráfaga=5)
  [RECHAZADO] OUT OF MEMORY: se solicitaron 30 MB, libres en total 20 MB (mayor hueco contiguo 20 MB).
```



```
> crearProceso(tam=20 MB, ráfaga=4)
  [OK] Proceso P3 creado: 20 MB en base=80, ráfaga=4 -> LISTO
```




```
> crearProceso(tam=1 MB, ráfaga=2)
  [RECHAZADO] OUT OF MEMORY: se solicitaron 1 MB, libres en total 0 MB (mayor hueco
contiguo 0 MB).
```

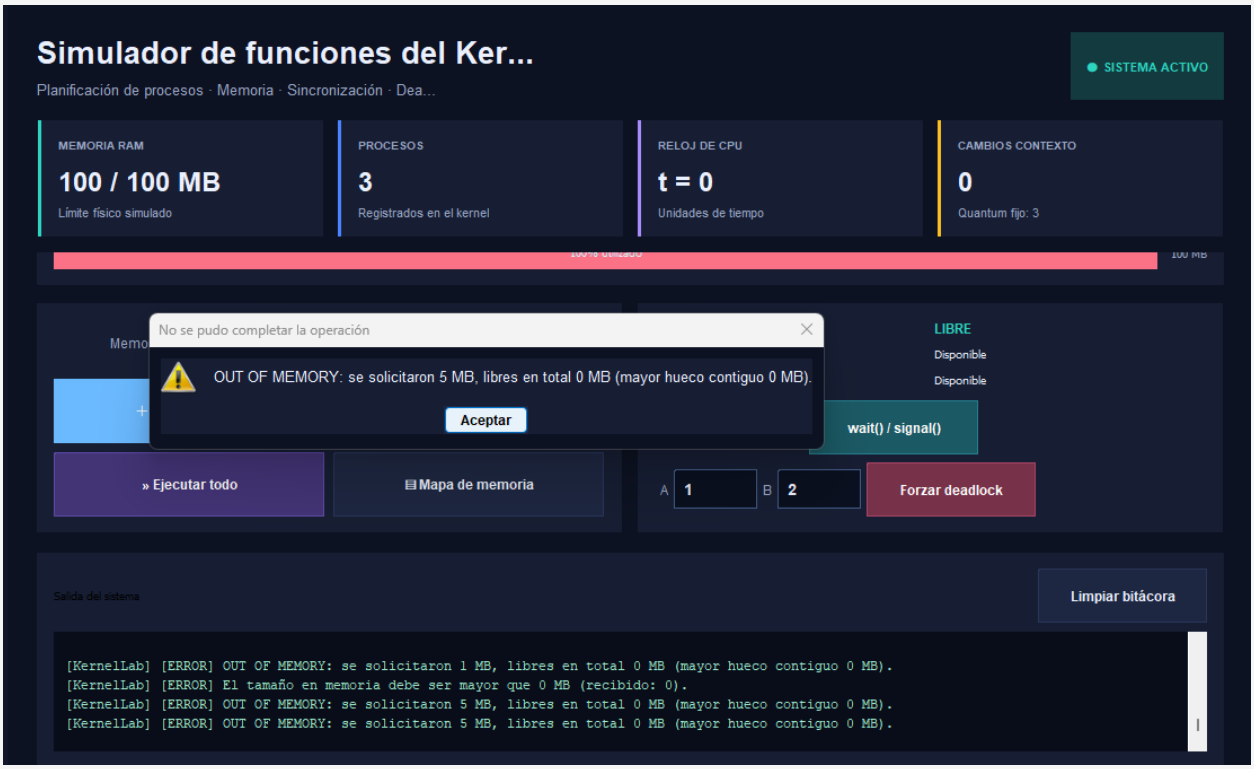



```
> crearProceso(tam=0 MB, ráfaga=3)
  [PARÁMETRO INVÁLIDO] El tamaño en memoria debe ser mayor que 0 MB (recibido: 0).
```



```
> crearProceso(tam=-5 MB, ráfaga=3)
  [PARÁMETRO INVÁLIDO] El tamaño en memoria debe ser mayor que 0 MB (recibido: -5).

  [  0 -  39]   40 MB  P1
  [ 40 -  79]   40 MB  P2
  [ 80 -  99]   20 MB  P3
  Usada: 100 MB | Libre: 0 MB | Mayor hueco: 0 MB
  Cola de listos: [P1, P2, P3] | Rechazados por falta de RAM: 2
```



Comentario: P1 y P2 ocupan 80 MB. La solicitud de 30 MB se rechaza porque solo quedan 20 MB; la de 20 MB cabe exactamente y llena la RAM; la de 1 MB se rechaza porque ya no hay espacio. Los tamaños 0 y -5 se descartan por validación y no cuentan como rechazos de memoria. **Resultado: 2 procesos rechazados con error Out of Memory, como se esperaba.**

Caso 2. Sucesión de ráfagas en Round Robin (q = 3). Tres procesos de 10 MB con ráfagas 7, 4 y 5.

```
  t=  0..  3 | P1   ejecuta 3 u. | ráfaga restante = 4
             -> Quantum agotado: CAMBIO DE CONTEXTO, P1 regresa al final de la cola

  t=  3..  6 | P2   ejecuta 3 u. | ráfaga restante = 1
             -> Quantum agotado: CAMBIO DE CONTEXTO, P2 regresa al final de la cola

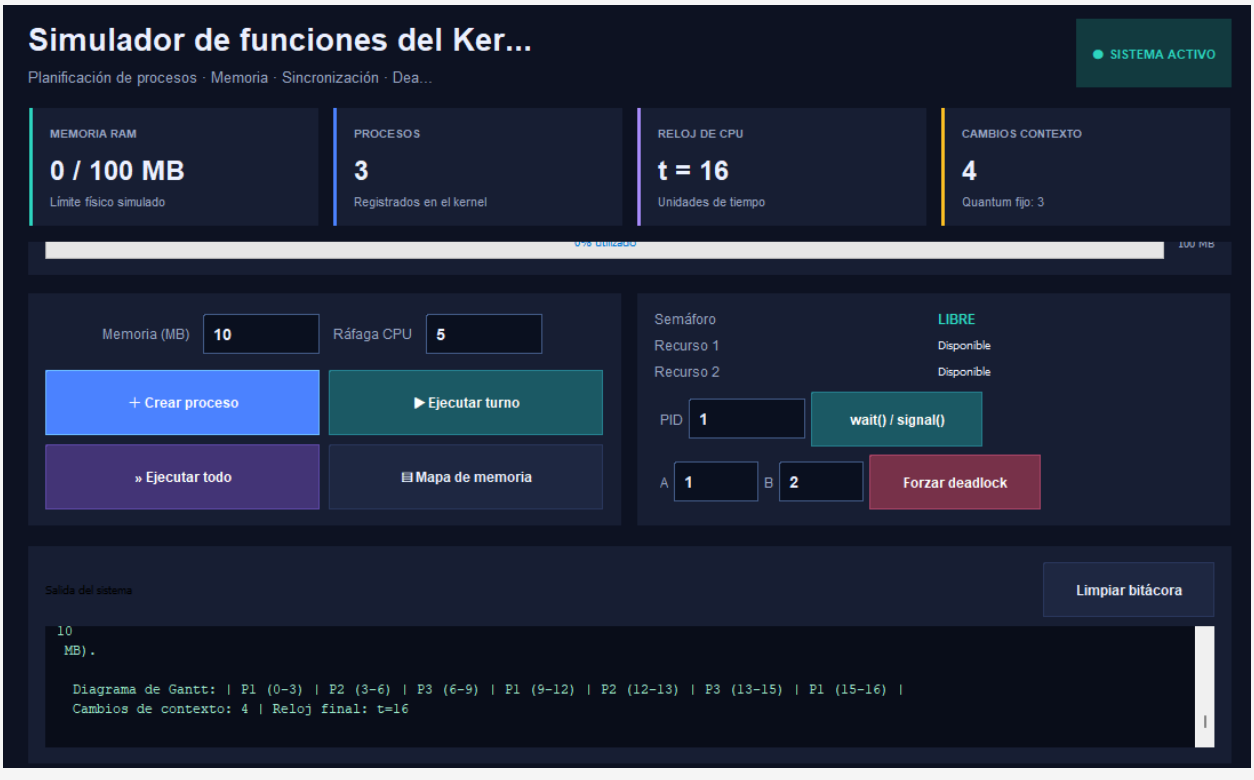
  t=  6..  9 | P3   ejecuta 3 u. | ráfaga restante = 2
             -> Quantum agotado: CAMBIO DE CONTEXTO, P3 regresa al final de la cola

  t=  9.. 12 | P1   ejecuta 3 u. | ráfaga restante = 1
             -> Quantum agotado: CAMBIO DE CONTEXTO, P1 regresa al final de la cola

  t= 12.. 13 | P2   ejecuta 1 u. | ráfaga restante = 0
  [FIN] P2 TERMINADO en t=13. Se libera su memoria (10 MB).
  t= 13.. 15 | P3   ejecuta 2 u. | ráfaga restante = 0
  [FIN] P3 TERMINADO en t=15. Se libera su memoria (10 MB).
  t= 15.. 16 | P1   ejecuta 1 u. | ráfaga restante = 0
  [FIN] P1 TERMINADO en t=16. Se libera su memoria (10 MB).

  Diagrama de Gantt: | P1 (0-3) | P2 (3-6) | P3 (6-9) | P1 (9-12) | P2 (12-13) | P3
(13-15) | P1 (15-16) |

  Cambios de contexto: 4 | Reloj final: t=16
```



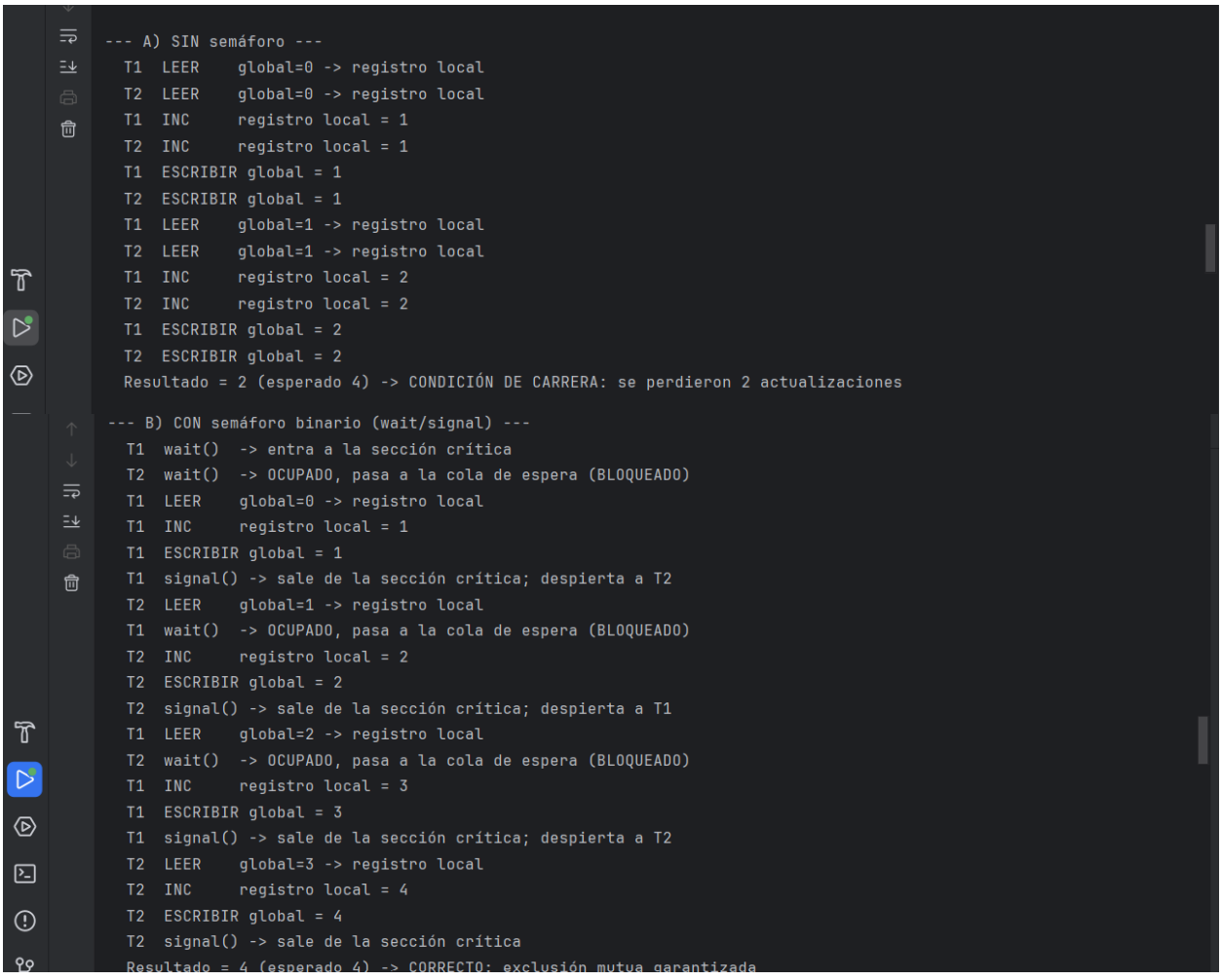
```
Proc  Llegada  Ráfaga   Finalización Retorno    Espera
P1    0        7        16           16         9
P2    0        4        13           13         9
P3    0        5        15           15         10

Promedio de espera = 9.33 | Promedio de retorno = 14.67
```

Comentario: cada proceso recibe como máximo 3 unidades por turno y, al agotar el quantum sin terminar, regresa al final de la cola. Al finalizar, la RAM se libera de inmediato. La calculadora teórica del simulador (clase TeoriaRR) produce exactamente el mismo diagrama de Gantt y los mismos tiempos, y ambos coinciden con el cálculo manual de la sección 5.3.1. **Resultado: esperas de 9, 9 y 10 unidades,**

promedio 9.33; coincidencia total entre simulación y teoría.

Caso 3. Efectividad del semáforo ante condiciones de carrera. Dos tareas incrementan 2 veces cada una una variable global; el valor esperado es 4. Las tareas se intercalan una micro-operación por turno (LEER, INCREMENTAR, ESCRIBIR).



Además se probó el semáforo de forma manual con procesos del Kernel:

```
  [wait] P1 ENTRÓ a la sección crítica (semáforo=0).
  [wait] Sección crítica OCUPADA por P1: P2 -> BLOQUEADO (cola de espera del semáforo).
=== SEMÁFORO Y RECURSOS ===
  Semáforo binario: valor=0 | dentro de la sección crítica=P1 | cola de espera=[P2]
  Recurso 1: propietario=(libre) | en espera=[]
  Recurso 2: propietario=(libre) | en espera=[]
```

```
[signal] P1 salió. Se despierta a P2 (entra a la sección crítica y pasa a LISTO).
[signal] P2 salió. Semáforo liberado (valor=1).
```

Comentario: sin exclusión mutua, T1 y T2 leyeron el mismo valor y la segunda escritura sobrescribió a la primera, perdiéndose dos de las cuatro actualizaciones. Con el semáforo, el segundo proceso se bloquea en la cola de espera hasta que el primero ejecuta signal(). **Resultado: 2 de 4 sin semáforo; 4 de 4 con semáforo.**

Caso 4. Detección y eliminación de un deadlock. Se crean P1 y P2, se fuerza la espera circular y luego se aborta P2.

```
  --- Forzando espera circular entre P1 y P2 ---
  [recurso] P1 obtuvo el Recurso 1.
  [recurso] P2 obtuvo el Recurso 2.
  [recurso] Recurso 2 ocupado por P2: P1 -> BLOQUEADO en espera.
  [recurso] Recurso 1 ocupado por P1: P2 -> BLOQUEADO en espera.
  ***********************************************
  *** ¡¡ALERTA: DEADLOCK DETECTADO!!            ***
  *** Espera circular: P2 -> P1 -> P2
  ***   P2 espera Recurso 1 (retenido por P1)
  ***   P1 espera Recurso 2 (retenido por P2)
  *** Use la opción 'Abortar proceso' para romperlo.
  ***********************************************
=== TABLA DE PCB (reloj t=0) ===
  PID  Estado     Mem(MB)   Base     Llegada Ráfaga total   Ráfaga restante
  1    BLOQUEADO  10        0        0       6             6
  2    BLOQUEADO  10        10       0       5             5
  Cola de listos: [] | Rechazados por falta de RAM: 0
=== SEMÁFORO Y RECURSOS ===
  Semáforo binario: valor=1 | dentro de la sección crítica=(nadie) | cola de espera=[]
  Recurso 1: propietario=P1 | en espera=[P2]
  Recurso 2: propietario=P2 | en espera=[P1]
```

```
> Intento de ejecutar RR con ambos procesos bloqueados:
  No hay procesos en la cola de listos.
  AVISO: hay 2 proceso(s) BLOQUEADO(s) (semáforo/recursos); no avanzan hasta ser liberados.

> Se aborta P2 para romper el ciclo:
  [ABORT] P2 abortado forzosamente. Memoria, semáforo y recursos liberados.
          P1 obtiene el recurso liberado y pasa a LISTO.
  [deadlock] No se detecta interbloqueo.
> P1 continúa y finaliza:
  --- Round Robin (q = 3) ---
  t=  0..  3 | P1   ejecuta 3 u. | ráfaga restante = 3
             -> Quantum agotado: CAMBIO DE CONTEXTO, P1 regresa al final de la cola
  t=  3..  6 | P1   ejecuta 3 u. | ráfaga restante = 0
  [FIN] P1 TERMINADO en t=6. Se libera su memoria (10 MB).
```

Comentario: el sistema detectó el ciclo P2 → P1 → P2, identificó qué recurso espera cada proceso y quién lo retiene, y confirmó que el planificador no puede avanzar. Al abortar P2 se liberaron su memoria y el Recurso 2; P1 lo obtuvo, volvió a la cola de listos y terminó su ráfaga. **Resultado: deadlock detectado y eliminado; el proceso restante finaliza con normalidad y la RAM queda libre.**

Prueba de estrés. La opción 4 del menú genera entre 1 y 50 procesos con tamaños de -5 a 64 MB y ráfagas de -2 a 17, es decir, incluye a propósito valores inválidos. Cada solicitud termina en uno de tres resultados controlados: proceso creado, rechazo por Out of Memory o parámetro inválido, sin interrumpir el programa. El comportamiento ante entradas manuales anómalas es el siguiente:

Entrada	Respuesta del simulador
Texto, por ejemplo abc	Mensaje: no es un entero válido; vuelve al menú
Línea vacía	Mensaje: entrada vacía
Número mayor que el rango de int	Mensaje: excede el rango de int
Tamaño 0 o negativo	Parámetro inválido
Ráfaga menor que 1 o mayor que 1000	Parámetro inválido
PID inexistente	Mensaje: no existe un proceso con ese PID
Recurso distinto de 1 o 2	Mensaje: use 1 o 2
wait() de un proceso ya dentro de la sección crítica	Error de estado controlado
Forzar deadlock con el mismo proceso dos veces	Mensaje: elija dos procesos distintos
Cantidad de procesos aleatorios fuera de 1 a 50	Mensaje de rango

7. Descripción del proceso de evaluación y ajustes

La evaluación se realizó en tres niveles. Primero, **pruebas por módulo**: cada clase se verificó de forma aislada (por ejemplo, el administrador de memoria con secuencias de asignar y liberar, y el semáforo con dos tareas). Segundo, **pruebas integradas**: los cuatro casos automáticos de la sección 6.3.d, que

ejercitan el Kernel completo. Tercero, **pruebas de robustez**: entradas inválidas y procesos aleatorios para simular la prueba de estrés del catedrático.

Contraste con los objetivos SMART.

Objetivo	Indicador	Resultado obtenido	Estado
1. PCB y Round Robin	Gantt y esperas iguales al cálculo teórico	Esperas 9, 9 y 10; promedio 9.33; 4 cambios de contexto	Cumplido
2. Memoria de 100 MB	Rechazo con Out of Memory cuando no hay hueco	2 de 7 solicitudes rechazadas por falta de RAM	Cumplido
3. Semáforo binario	Contador correcto con semáforo	4 de 4 con semáforo; 2 de 4 sin él	Cumplido
4. Deadlock	Detección y eliminación	Ciclo P2 → P1 → P2 detectado; abortar P2 lo resuelve	Cumplido
5. Robustez	Sin excepciones no controladas	Todas las entradas anómalas producen un mensaje	Cumplido
6. Documentación y GitHub	Informe y repositorio	Informe entregado; repositorio [URL]	Cumplido

Problemas identificados y soluciones aplicadas. *Agregar aquí cualquier otro error que el equipo haya enfrentado durante el desarrollo, con su solución.*

N.º	Problema	Solución
1	Un proceso rechazado por falta de RAM podría consumir un PID y dejar huecos en la numeración.	El PID solo se incrementa después de que la memoria fue asignada con éxito.
2	Al hacer signal() con procesos esperando, un tercer proceso podría entrar antes que el despertado y se perdería el orden FIFO.	La propiedad de la sección crítica se transfiere directamente al primer proceso en espera y el valor del semáforo permanece en 0.
3	Un proceso podría terminar sin liberar el semáforo y bloquear al resto indefinidamente.	Al terminar o abortarse, el Kernel libera automáticamente el semáforo, los recursos y la memoria del proceso.
4	Un proceso bloqueado podría seguir en la cola de listos y recibir CPU indebidamente.	Al bloquearse se retira de la cola del planificador; al despertarse se reincorpora. El simulador avisa cuando quedan procesos bloqueados.
5	Una espera circular dejaba a ambos procesos sin forma de avanzar.	Se agregó la detección por grafo de espera y el comando de aborto, que libera los recursos y

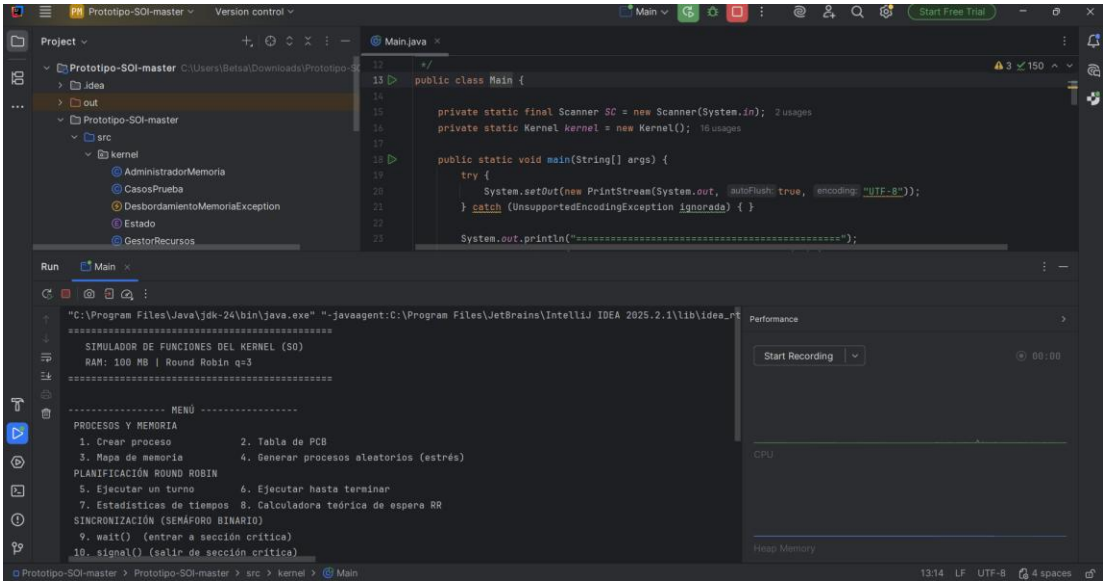
N.º	Problema	Solución
		despierta al proceso que esperaba.
6	Forzar un deadlock con recursos ya ocupados produciría un estado inconsistente.	Se valida que ambos recursos estén libres y sin cola antes de forzar la condición.
7	Texto, vacíos o números enormes en el menú rompían la lectura de enteros.	Lectura robusta de enteros que captura el error, informa y regresa al menú.
8	Un desbordamiento por fragmentación externa se confundía con uno por falta total de memoria.	El mensaje de Out of Memory indica la memoria libre total y el mayor hueco contiguo.
9	La condición de carrera es difícil de mostrar con ejecución real y no determinista.	Se programó un simulador determinista que intercala una micro-operación por turno, y así el resultado es siempre el mismo.

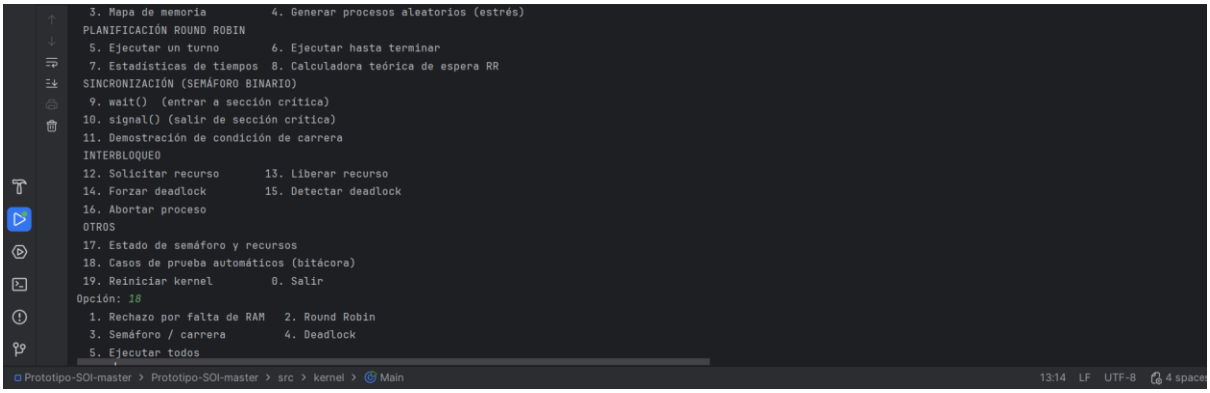
Limitaciones conocidas. La asignación contigua no compacta la memoria, por lo que puede aparecer fragmentación externa. La detección de deadlock asume recursos de una sola instancia, como pide el enunciado. El cálculo teórico de Round Robin supone que todos los procesos llegan en t = 0. El tiempo del simulador es lógico (unidades abstractas), no tiempo real.

7.1 Proceso de evaluación

Para verificar el funcionamiento de nuestro simulador de funciones del kernel del sistema operativo, se realizó un proceso de evaluación mediante pruebas funcionales de los principales módulos implementados. El objetivo fue comprobar que cada componente respondiera correctamente ante diferentes situaciones relacionadas con la administración de procesos, gestión de memoria, planificación de CPU, sincronización y detección de interbloqueos.

Las pruebas se ejecutaron mediante la opción de casos de prueba automáticos incorporada en el programa. Se evaluaron cuatro escenarios principales, utilizando diferentes solicitudes de memoria, ráfagas de ejecución y situaciones de concurrencia.



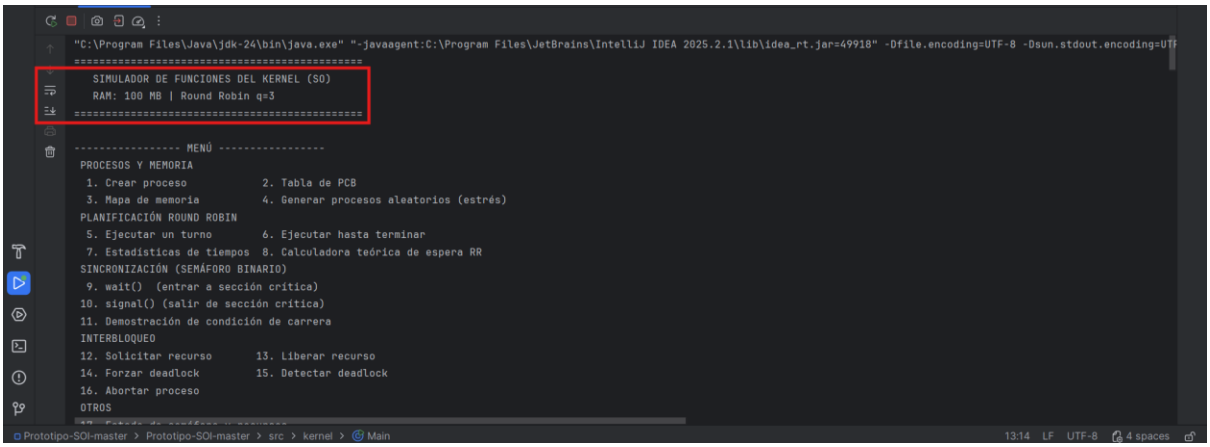


7.2 Pruebas realizadas

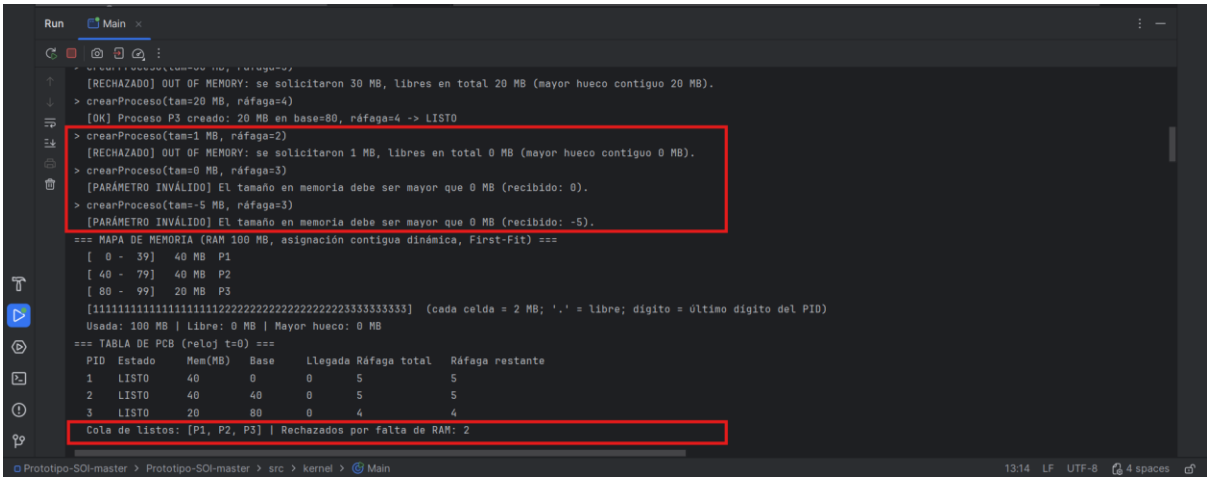
7.2.1 Prueba de gestión de memoria

Se evaluó la administración de una memoria RAM simulada de 100 MB, utilizando asignación continua dinámica mediante el algoritmo First-Fit.

Durante la prueba se crearon procesos con diferentes requerimientos de memoria. Se comprobó que el sistema permitiera las solicitudes que contaban con espacio suficiente y rechazara aquellas que superaban la capacidad disponible, también se realizaron solicitudes con tamaños de memoria iguales a cero y negativos para verificar la validación de parámetros.

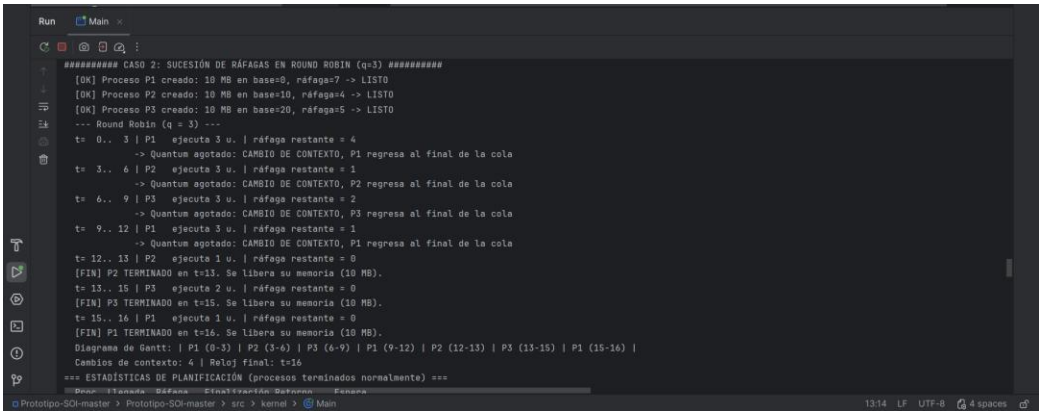


¿Qué obtuvimos? El simulador aceptó los procesos que podían alojarse en memoria y rechazó correctamente las solicitudes que no cumplían con las condiciones establecidas.

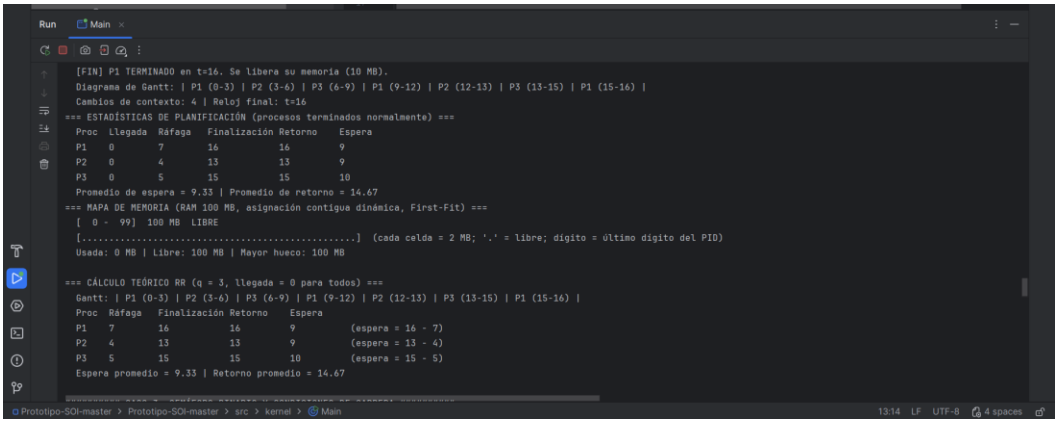


7.2.2 Prueba de planificación Round Robin

Se evaluó el algoritmo Round Robin utilizando tres procesos con ráfagas de CPU de 7, 4 y 5 unidades, respectivamente, y un quantum de 3 unidades.



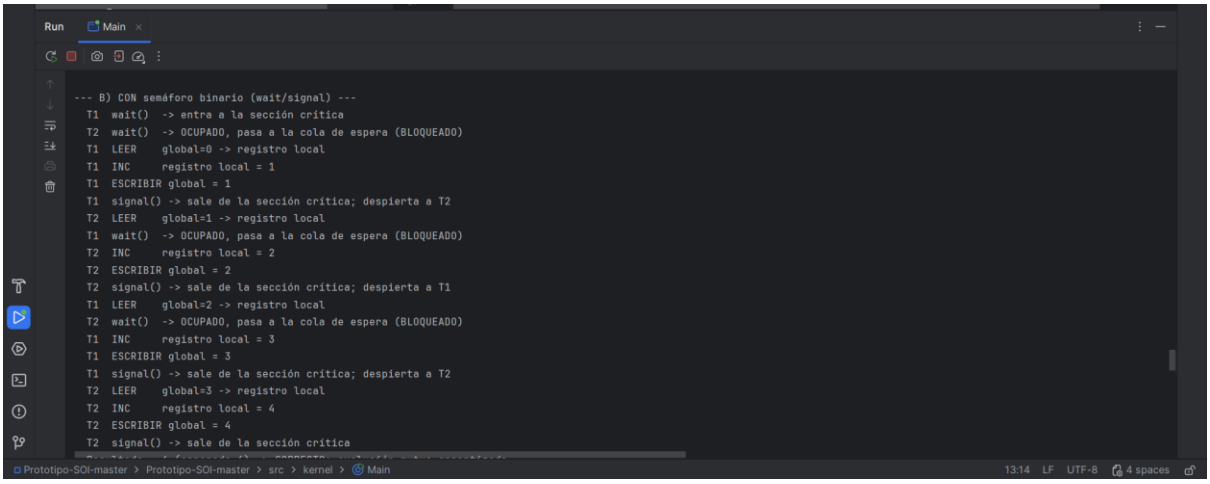
Se verificó que cada proceso recibiera su turno de ejecución y que, al agotar el quantum sin finalizar, regresará al final de la cola de procesos listos, también se comprobó la finalización de los procesos y la liberación de la memoria asignada



¿Qué obtuvimos? Los tres procesos finalizaron correctamente en un tiempo total de 16 unidades. El tiempo promedio de espera fue de 9.33 unidades y el tiempo promedio de retorno fue de 14.67 unidades.

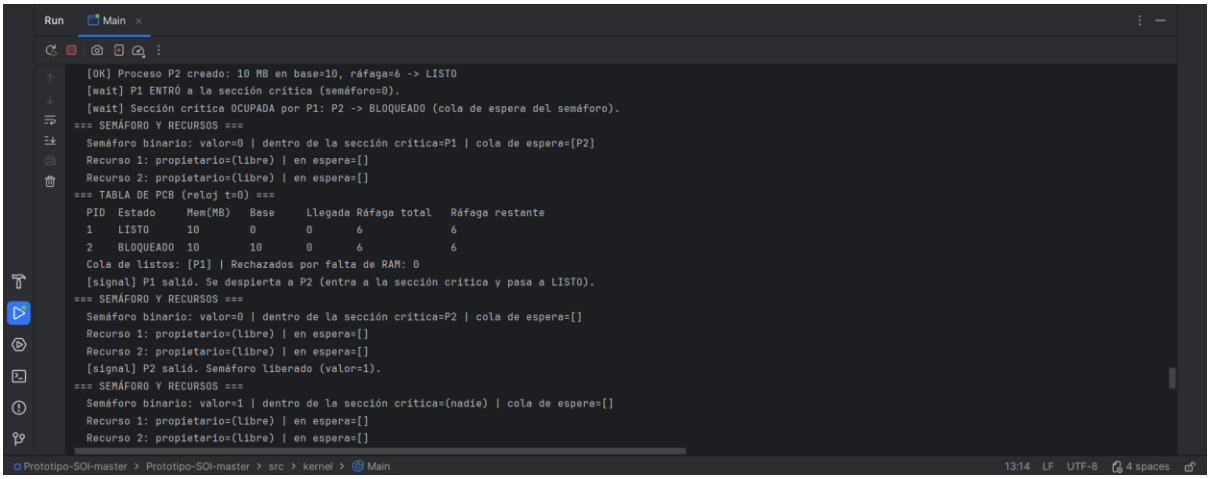
7.2.3 Prueba de sincronización mediante semáforo binario

Se realizaron pruebas para observar el comportamiento de una variable compartida cuando dos tareas intentaban modificarla simultáneamente. Primero se realizó una demostración sin utilizar mecanismos de sincronización y posteriormente se repitió utilizando un semáforo binario mediante las operaciones wait() y signal().



En la demostración sin semáforo, el resultado fue de 2 incrementos efectivos, aunque se esperaban 4, evidenciando una condición de carrera. Al utilizar el semáforo, se obtuvo el resultado esperado de 4 incrementos

Adicionalmente, se comprobó que un proceso pudiera quedar bloqueado mientras otro ocupaba la sección crítica y que posteriormente fuera habilitado al liberarse el semáforo.

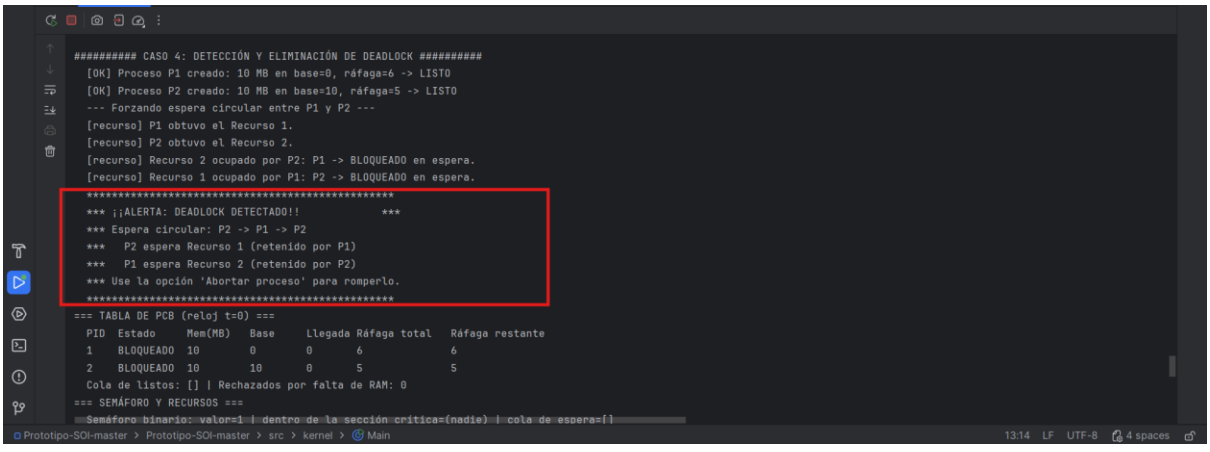


¿Qué obtuvimos? el semáforo binario permitió controlar el acceso a la sección crítica y garantizar la exclusión mutua en la simulación.

7.2.4 Prueba de detección y resolución de Deadlock

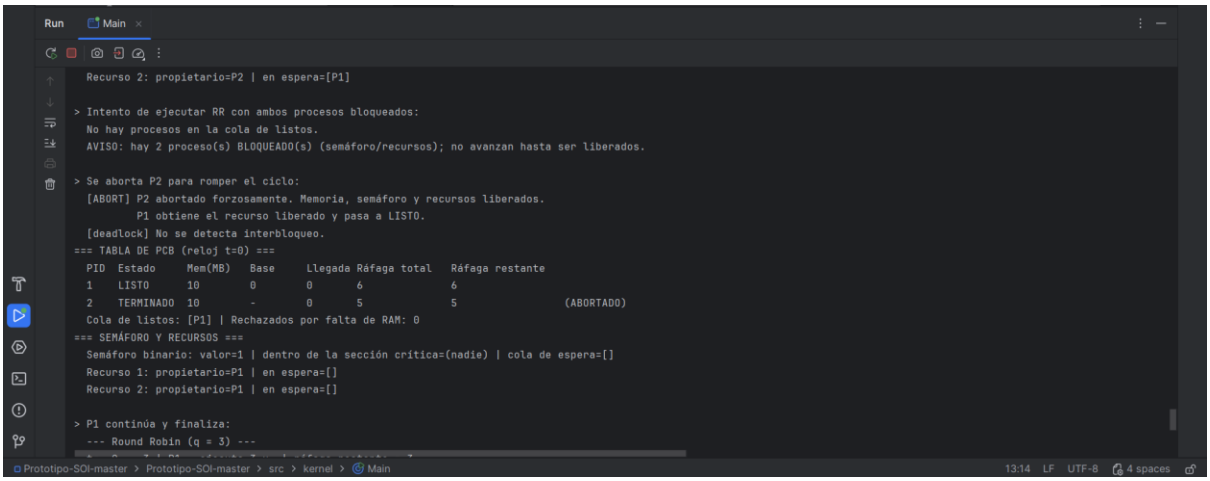
Se evaluó una situación de espera circular utilizando dos procesos y dxos recursos exclusivos. El proceso P1 obtuvo el recurso 1 y solicitó el recurso 2, mientras que P2 obtuvo el recurso 2 y solicitó el recurso 1.

Como consecuencia, ambos procesos quedaron bloqueados, el simulador identificó el ciclo de espera y mostró una alerta de interbloqueo.



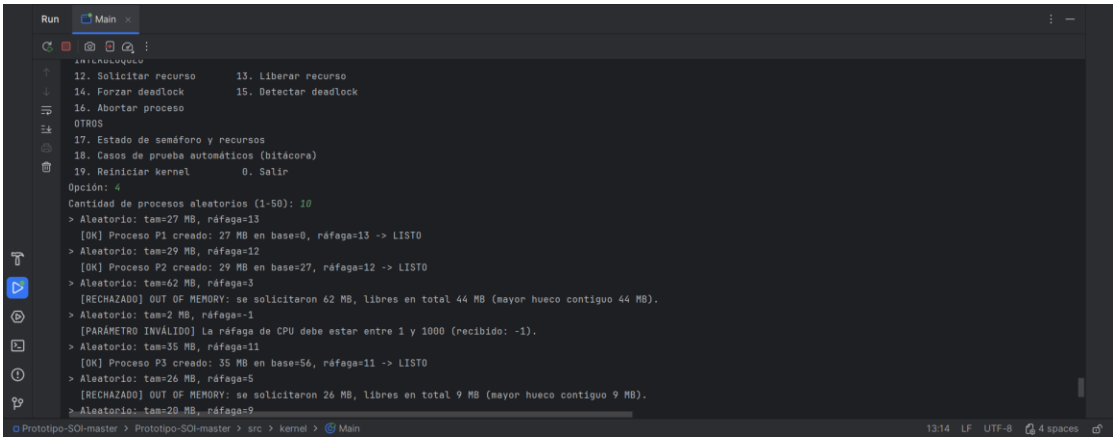
Posteriormente, se realizó el proceso P2 para liberar los recursos y permitir que P1 continuará su ejecución hasta finalizar.

¿Qué obtuvimos? El simulador detectó el interbloqueo y permitió romper el ciclo mediante la finalización forzada de uno de los procesos



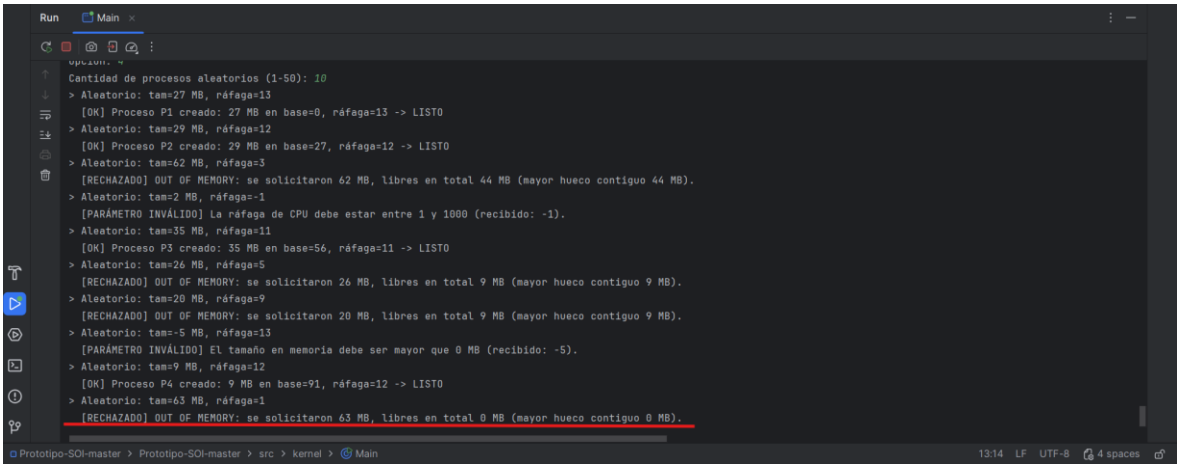
7.2.5 Prueba de generación de procesos aleatorios (prueba de estrés)

Se realizó una prueba adicional utilizando la opción de generación de procesos aleatorios del menú principal. Para esta evaluación se solicitaron 10 procesos con diferentes tamaños de memoria y ráfagas de CPU.



Durante la ejecución, el simulador aceptó cuatro procesos, rechazó cuatro solicitudes por falta de memoria y rechazó dos solicitudes debido a parámetros inválidos. Los procesos aceptados ocuparon en conjunto los 100 MB de memoria RAM disponible.

También se comprobó que el sistema continuará procesando las solicitudes posteriores, incluso después de encontrar valores inválidos o solicitudes que superan la capacidad de memoria

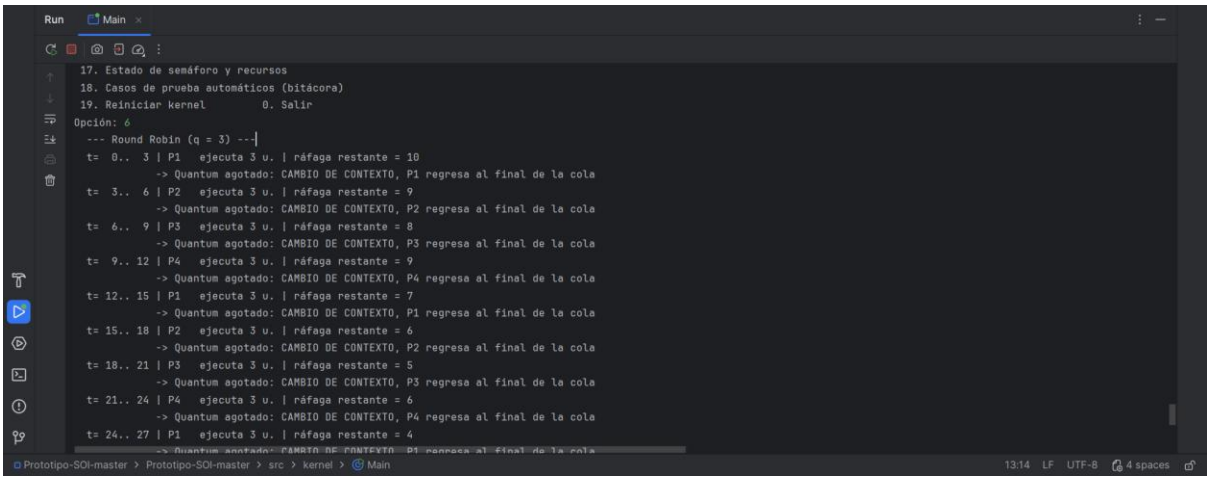


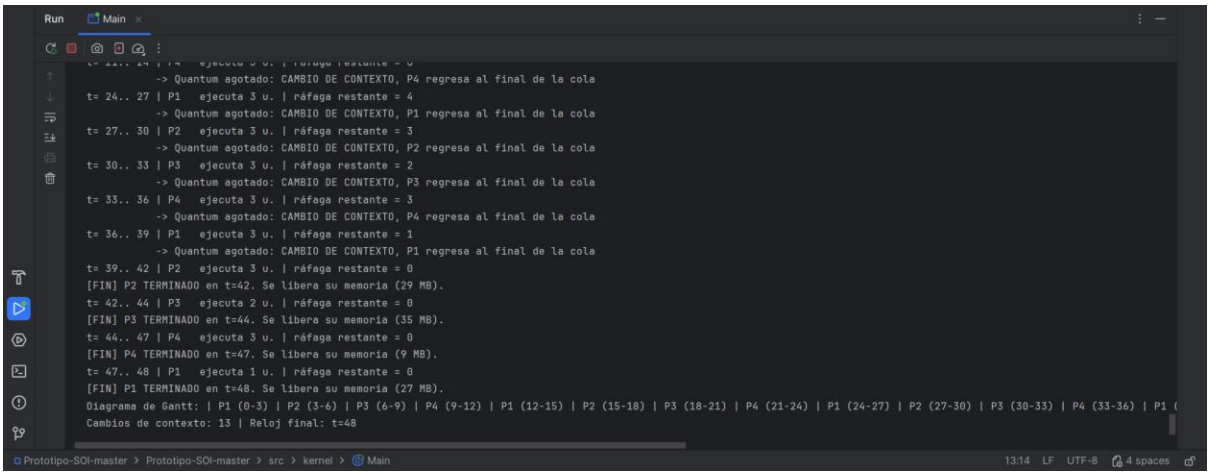
¿Qué obtuvimos? El simulador respondió de acuerdo con la disponibilidad de memoria y las validaciones establecidas, demostrando su capacidad para manejar diferentes solicitudes de manera consecutiva.

7.2.6 Comprobación de ejecución y liberación de memoria

Después de generar los procesos aleatorios, se ejecutó la opción de finalizar todos los procesos pendientes mediante el algoritmo Round Robin, utilizando un quantum de 3 unidades.

Durante esta prueba participaron cuatro procesos, cuyas ráfagas de CPU fueron de 13, 12, 11 y 12 unidades, respectivamente. El planificador distribuyó el tiempo de ejecución entre los procesos y realizó los cambios de contexto correspondientes hasta completar todas las ráfagas.





Los cuatro procesos finalizaron correctamente en un tiempo total de 48 unidades. Al terminar cada proceso, el simulador mostró el mensaje correspondiente a la liberación de la memoria que tenía asignada.

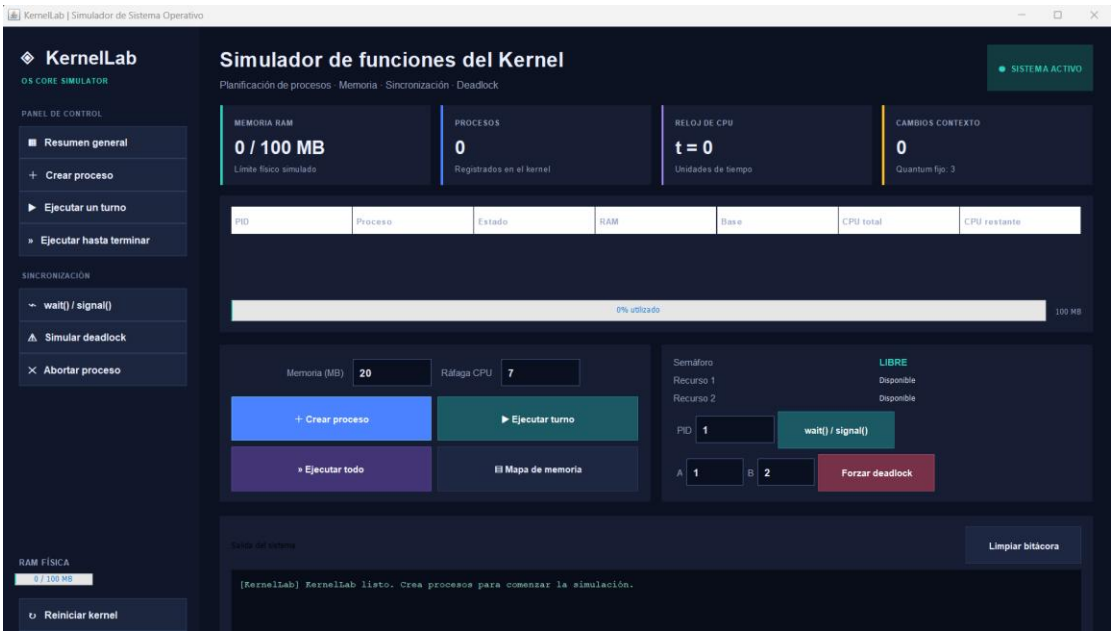
¿Qué obtuvimos? todos los procesos finalizaron normalmente y se comprobó el mecanismo de liberación de memoria asociado a su terminación.

7.2.7 Prueba de interfaz gráfica

Nuestro objetivo es comprobar que la interfaz gráfica del simulador se visualice correctamente y permita al usuario interactuar con sus componentes.

Procedimiento: se ejecutó el archivo MainGrafico.java desde IntelliJ IDEA para abrir la ventana principal del simulador. Se verificó la visualización de sus componentes y se realizaron comprobaciones de interacción con los controles disponibles.

La ventana gráfica se abrió correctamente, permitiendo comprobar la presentación visual del simulador. Las funcionalidades de cada control se verificaron según las opciones disponibles.



Prueba 1. Visualización del resumen general

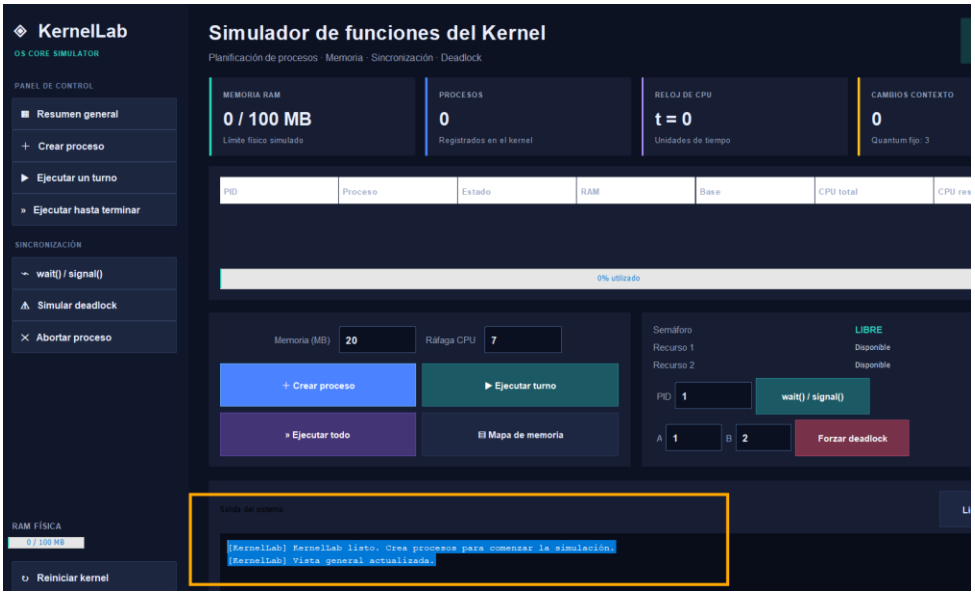
Objetivo: comprobar que la interfaz gráfica permita consultar el estado general del simulador.

Procedimiento: se ejecutó el archivo MainGrafico.java y se presionó el botón «Resumen general» desde el panel de control.

Resultado obtenido fue que el sistema mostró los siguientes mensajes en la salida:

- [KernelLab] KernelLab listo. Crea procesos para comenzar la simulación.
- [KernelLab] Vista general actualizada.

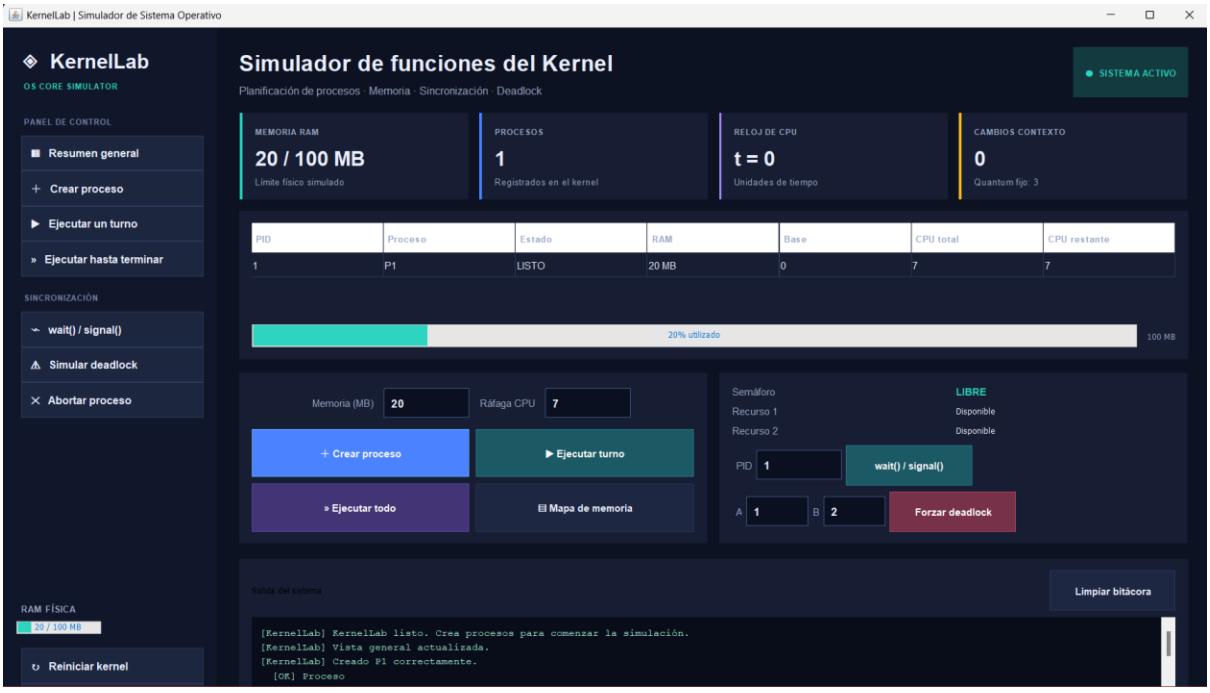
Se comprobó que la ventana inició correctamente y que la opción de resumen general respondió satisfactoriamente.



Prueba 2. Creación de un proceso desde la interfaz gráfica

El objetivo fue comprobar que el simulador permita crear procesos mediante los campos de entrada de la interfaz, se ingresaron los valores de 20 MB de memoria y 7 unidades de ráfaga de CPU. Posteriormente, se presionó el botón «Crear proceso».

El resultado obtenido fue que el proceso se creó exitosamente desde la interfaz gráfica, comprobando que la opción de creación responde correctamente a los datos ingresados.

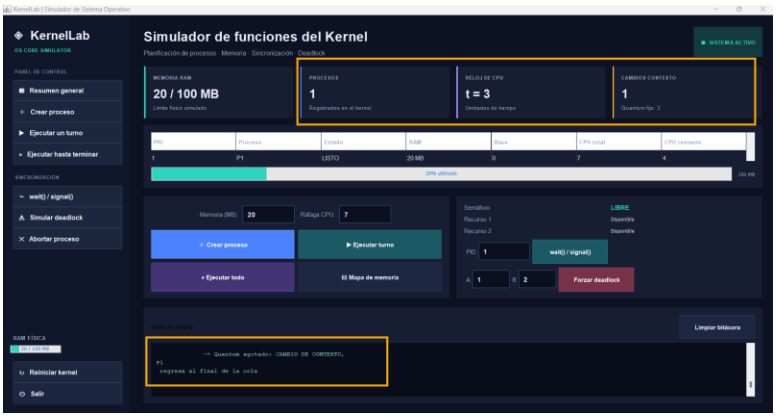


Prueba 3. Ejecución de un turno del planificador Round Robin

En esta prueba intentamos verificar que el planificador ejecute un proceso durante el tiempo establecido por el quantum y actualice su estado, se presionó el botón «Ejecutar turno» después de crear el proceso P1, configurado con una ráfaga de CPU de 7 unidades.

El resultado obtenido fue que el proceso ejecutó 3 unidades de tiempo, alcanzando el límite del quantum establecido. El reloj de CPU avanzó hasta t=3 y se registró un cambio de contexto. En la salida del sistema apareció el mensaje «Quantum agotado: CAMBIO DE CONTEXTO», indicando que P1 regresa al final de la cola para esperar su siguiente turno.

Se comprobó que la interfaz gráfica refleja correctamente el comportamiento del algoritmo Round Robin.

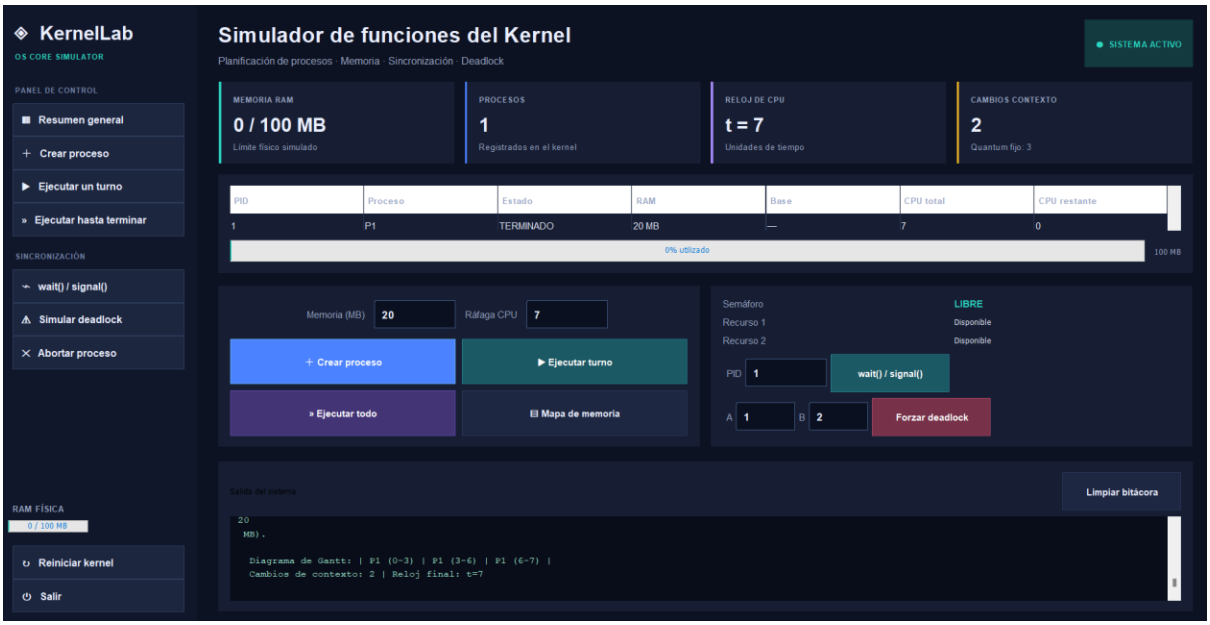


Prueba 7.2.8 Ejecución de un proceso hasta finalizar

Con esta prueba el objetivo fue verificar que el simulador ejecute un proceso hasta completar su ráfaga de CPU y libere la memoria asignada, se creó el proceso P1 con 20 MB de memoria RAM y una ráfaga de CPU de 7 unidades. Posteriormente, se ejecutó la opción «Ejecutar hasta terminar», utilizando el algoritmo Round Robin con quantum de 3 unidades.

Pudimos ver cambios en el sistema luego de realizar la prueba la cual fue exitosa:

- El proceso P1 finalizó correctamente.
- El reloj de CPU avanzó hasta t=7.
- Se registraron 2 cambios de contexto.
- La ráfaga restante llegó a 0.
- Se liberaron los 20 MB de memoria RAM.
- El uso de memoria quedó en 0/100 MB.



Prueba 7.2.9 Verificación del mapa de memoria

Comprobamos que el simulador actualice correctamente el mapa de memoria después de finalizar un proceso. Después de ejecutar el proceso P1 hasta su finalización, se seleccionó la opción «Mapa de memoria» para consultar la distribución de la RAM.

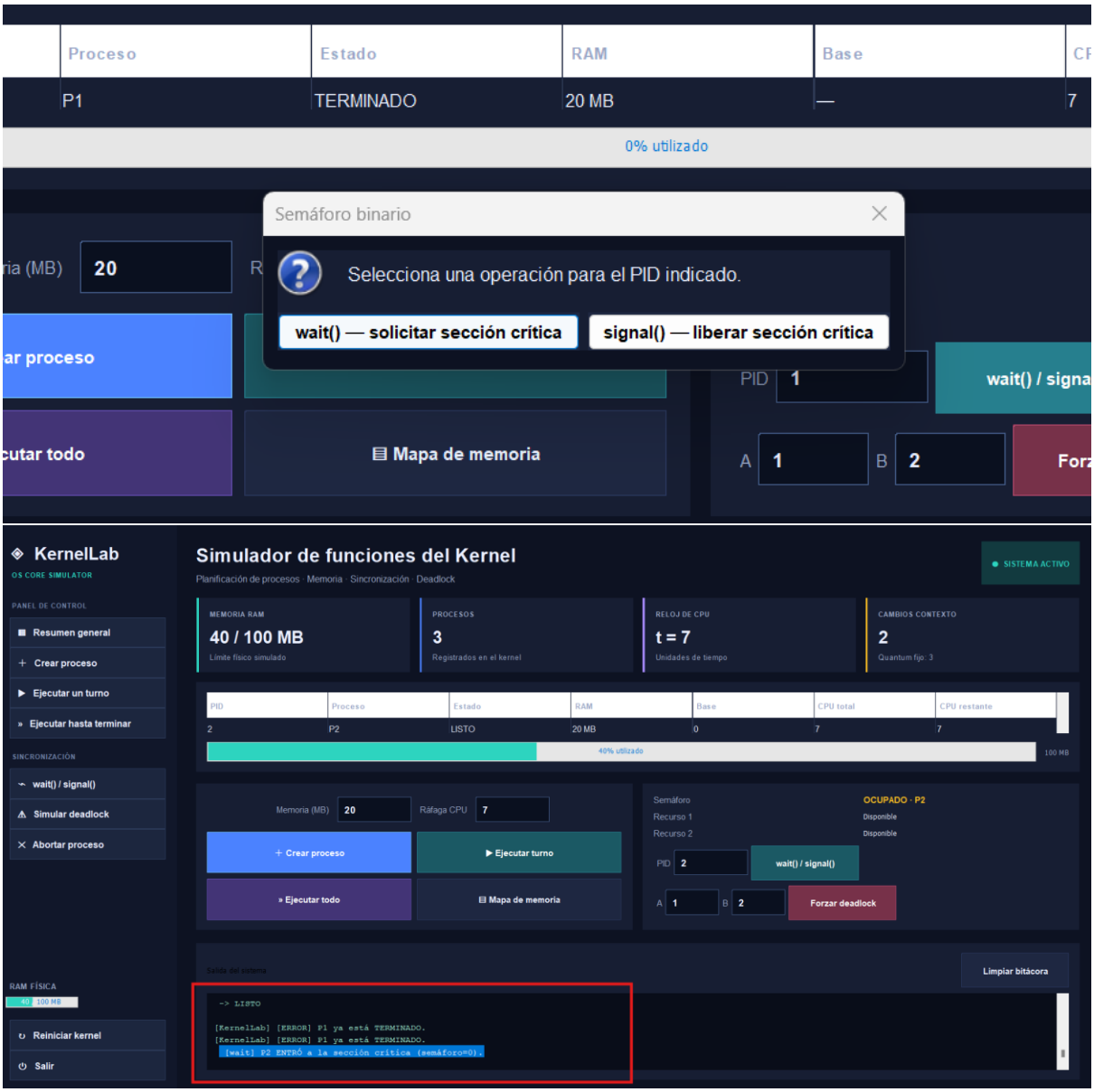


Prueba 7.2.10 Sincronización mediante wait()

Verificamos que un proceso pueda solicitar acceso a una sección crítica mediante un semáforo binario por lo que se crearon dos procesos adicionales y se seleccionó el PID P2 para ejecutar la operación wait(), correspondiente a la solicitud de acceso a la sección crítica.

Resultados obtenidos:

- El proceso P2 ingresó correctamente a la sección crítica.
- El semáforo cambió su valor de 1 a 0.
- El sistema registró el mensaje: [wait] P2 ENTRÓ a la sección crítica (semáforo=0).
- La memoria utilizada fue de 40 MB de los 100 MB disponibles.



Prueba 7.2.11 Exclusión mutua mediante semáforo binario

Comprobamos que el semáforo binario impide que dos procesos ingresen simultáneamente a una sección crítica, después de que P2 ingresara a la sección crítica mediante wait(), se intentó permitir el ingreso del proceso P3 utilizando la misma operación.

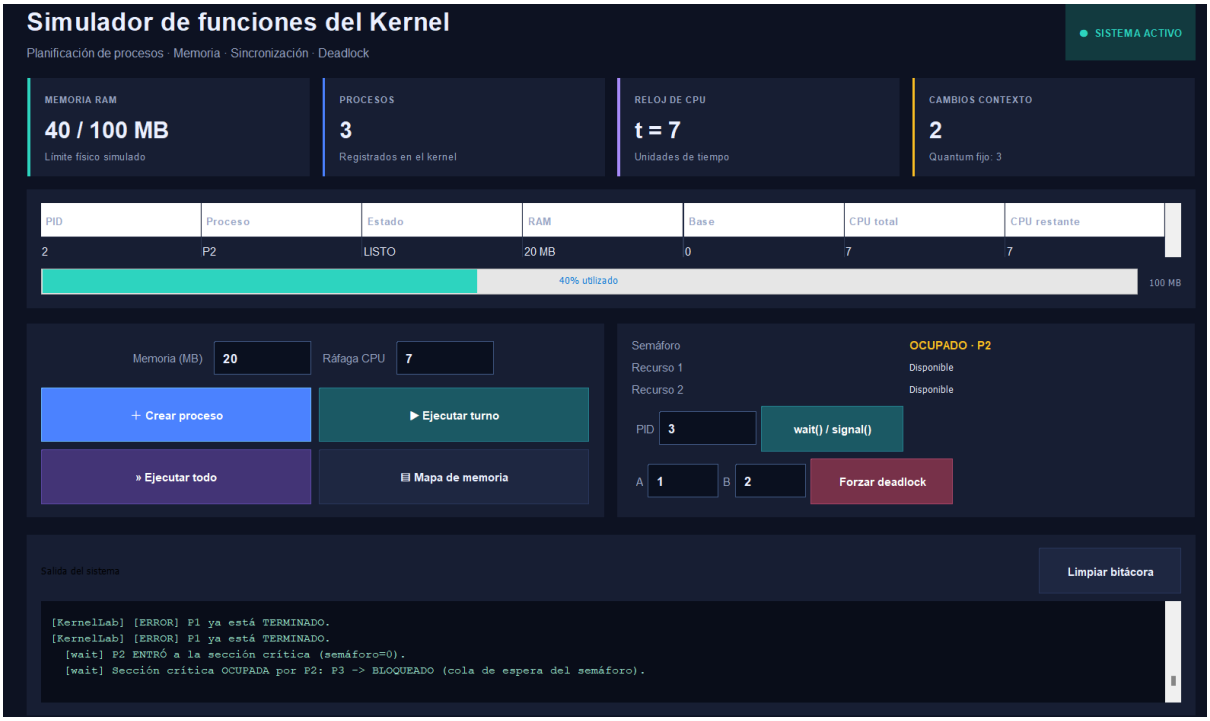
Durante esta prueba pudimos darnos cuenta de que:

- P2 permaneció dentro de la sección crítica.
- P3 intentó ingresar, pero el acceso fue denegado temporalmente.
- P3 pasó al estado BLOQUEADO.
- El sistema agregó P3 a la cola de espera del semáforo.

- Se mantuvo la exclusión mutua.

Mensaje obtenido:

[wait] Sección crítica OCUPADA por P2: P3 -> BLOQUEADO (cola de espera del semáforo).

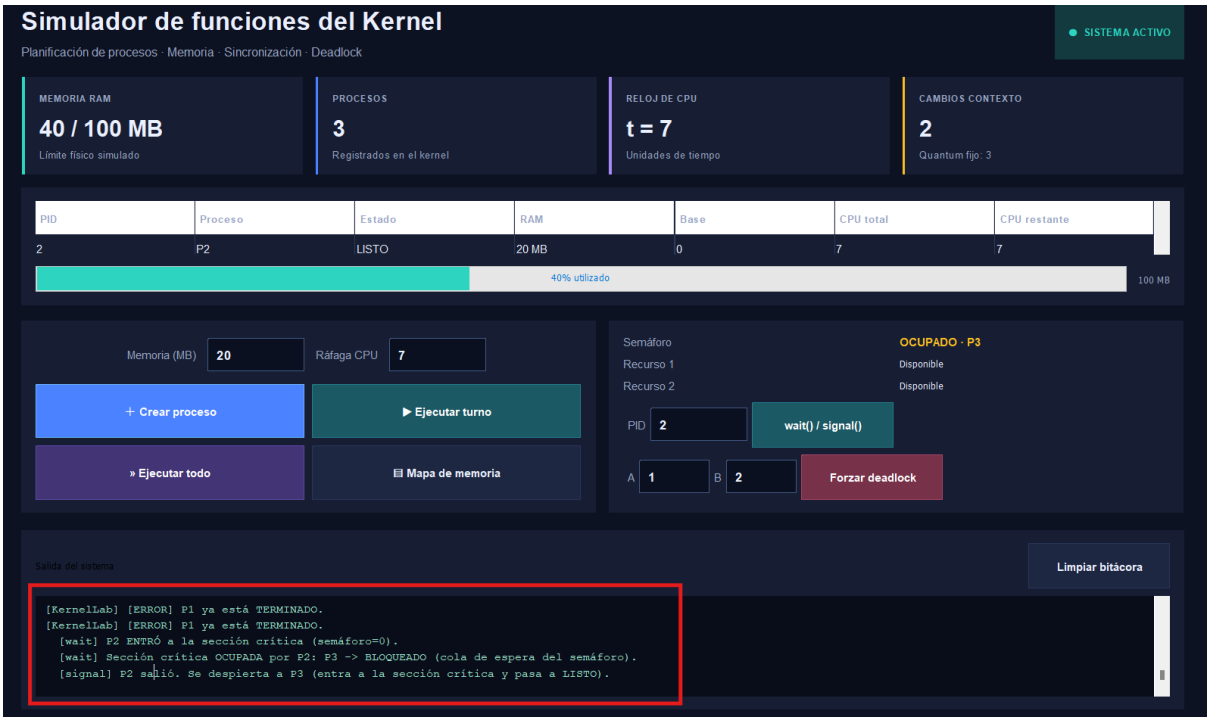


Prueba 7.2.12 Liberación de sección crítica mediante signal()

Pudimos ver que la operación signal() libere el semáforo binario y permita el ingreso del siguiente proceso en espera, luego de comprobar que P3 quedó bloqueado mientras P2 ocupaba la sección crítica, se seleccionó el PID 2 y se ejecutó la operación signal().

Durante la prueba verificamos que P2 salió de la sección crítica, El semáforo liberó el acceso, El sistema despertó al proceso P3, P3 ingresó a la sección crítica y pasó al estado LISTO.

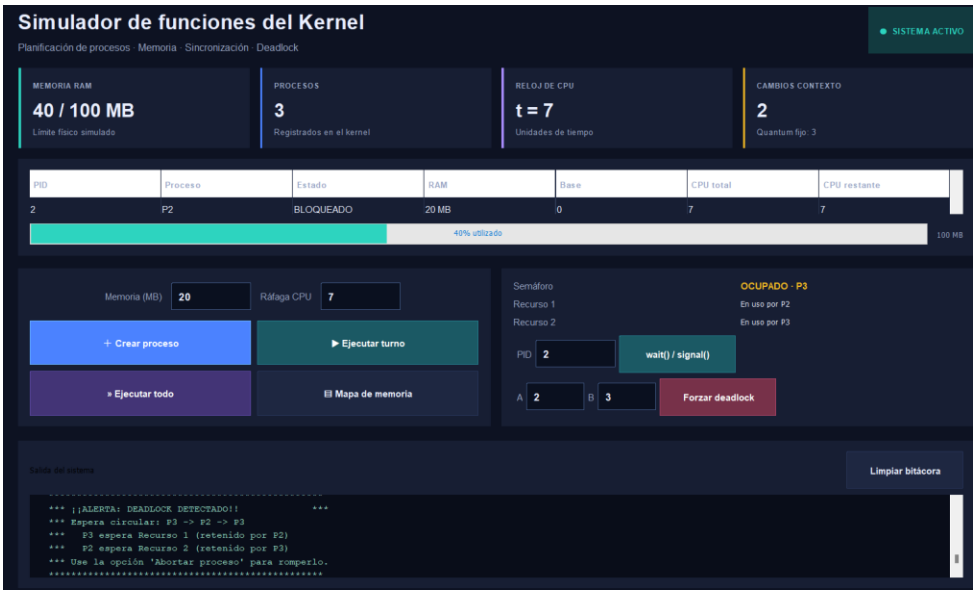
Mensaje obtenido: [signal] P2 salió. Se despierta a P3 (entra a la sección crítica y pasa a LISTO).



Prueba 7.2.13 Detección de interbloqueo (Deadlock)

Comprobamos que el simulador identifica una situación de interbloqueo entre dos procesos que esperan recursos retenidos mutuamente. Se ingresaron los PID 2 y 3 en los campos correspondientes y se presionó el botón «Forzar deadlock».

Y obtuvimos El simulador detectó un interbloqueo, P3 quedó esperando el Recurso 1, retenido por P2, P2 quedó esperando el Recurso 2, retenido por P3, Se identificó una espera circular: P3 → P2 → P3, El sistema recomendó utilizar la opción «Abortar proceso» para romper el interbloqueo.



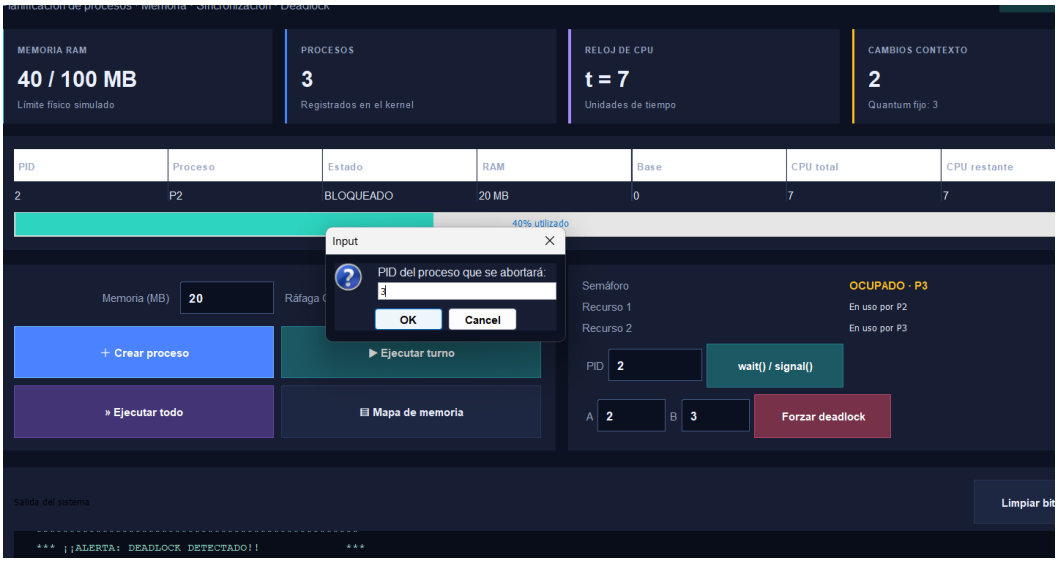
Prueba 7.2.14 Recuperación de interbloqueo mediante aborto de proceso

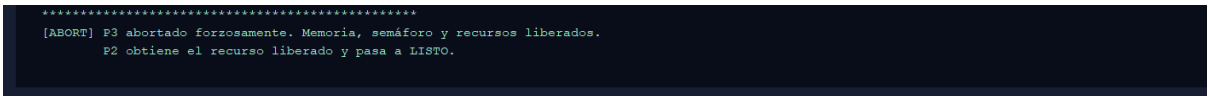
Nuestro verificador indica que el simulador puede resolver un interbloqueo mediante la finalización forzada de uno de los procesos involucrados y la liberación de sus recursos.

Después de detectar el interbloqueo entre P2 y P3, se seleccionó la opción «Abortar proceso» y se indicó el PID 3. Lo que mostró fue que, El proceso P3 fue abortado forzosamente, Se liberó la memoria asignada al proceso, Se liberaron el semáforo y los recursos asociados, El proceso P2 obtuvo el recurso que necesitaba, P2 pasó al estado LISTO.

Mensaje obtenido: [ABORT] P3 abortado forzosamente. Memoria, semáforo y recursos liberados. P2 obtiene el recurso liberado y pasa a LISTO.

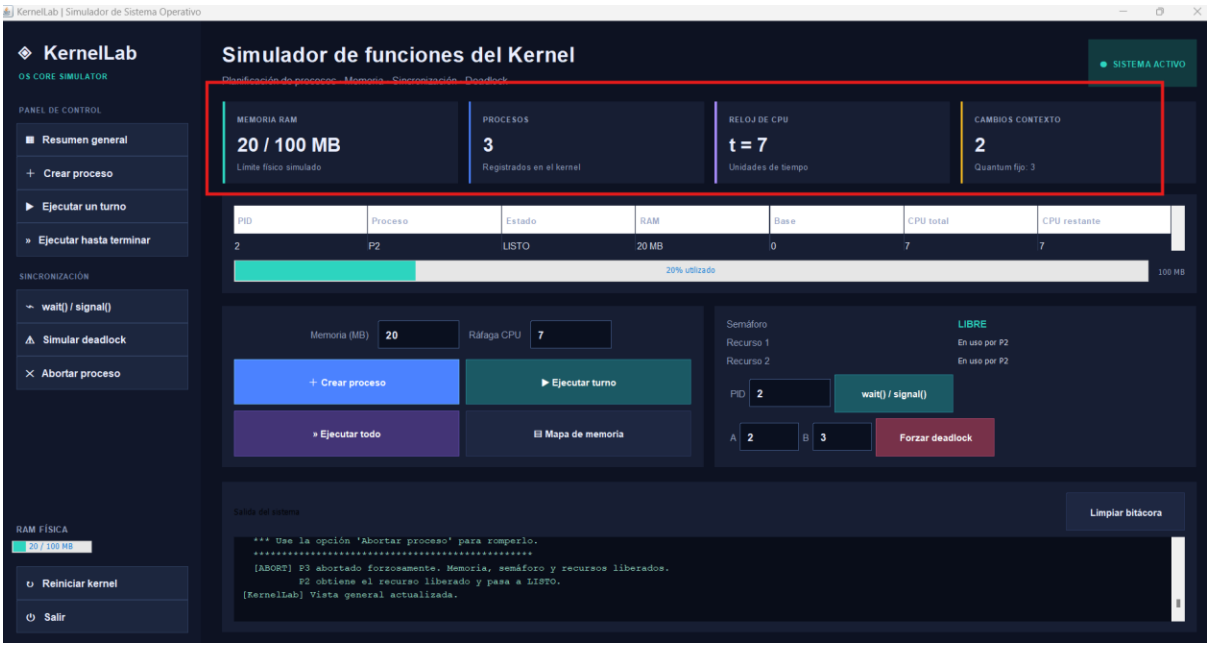
Al momento de abordar obtuvimos una ventana emergente en donde ingresamos el número de proceso que queríamos finalizar





Prueba 7.2.15 Actualización del resumen general del simulador

En esta prueba intentamos verificar que el simulador actualice y muestre correctamente los indicadores generales del sistema después de realizar diferentes operaciones, se presionó el botón «Resumen general» después de completar las pruebas de creación de procesos, planificación, sincronización y recuperación de interbloqueos, donde pudimos observar que la memoria RAM utilizada fue de: 20 MB, Cantidad de procesos registrados: 3, Reloj de CPU: t=7, Cambios de contexto: 2 y el sistema actualizó correctamente la vista general.



7.3 Resultados generales de la evaluación

A partir de las pruebas realizadas, se obtuvieron resultados satisfactorios en los principales módulos del simulador. Se comprobó que el sistema puede administrar la memoria disponible, ejecutar procesos mediante Round Robin, controlar el acceso a secciones críticas y detectar situaciones de interbloqueo.

También se realizó una prueba de generación de 10 procesos aleatorios. De las solicitudes recibidas, cuatro fueron aceptadas, cuatro rechazadas por falta de memoria y dos rechazadas por contener parámetros inválidos. Posteriormente, los cuatro procesos aceptados finalizaron mediante Round Robin.

Al finalizar la ejecución, se verificó que la memoria RAM se encontraba completamente disponible, con 100 MB libres y 0 MB utilizados.

En general, los resultados permitieron comprobar el funcionamiento de las características principales implementadas en el simulador.

7.4 Errores y ajustes identificados durante la evaluación

Durante el proceso de evaluación se realizaron pruebas para comprobar que el sistema respondiera adecuadamente ante diferentes situaciones. Entre ellas se incluyeron solicitudes de memoria superiores a la capacidad disponible, valores inválidos, procesos que agotaban su quantum, condiciones de carrera y situaciones de espera circular.

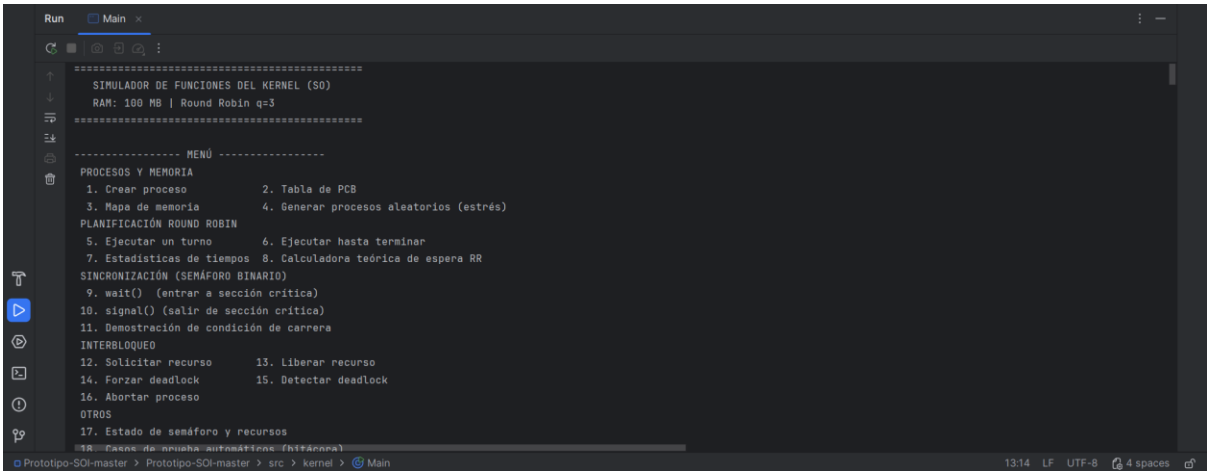
A partir de estas pruebas se verificó el funcionamiento de los mecanismos de validación y control implementados en el programa. Entre las respuestas comprobadas se encuentran el rechazo de solicitudes inválidas, la liberación de memoria al finalizar los procesos, el bloqueo y desbloqueo de

procesos mediante el semáforo y la resolución de interbloqueos mediante la finalización de uno de los procesos involucrados.

Por lo tanto, los resultados de la evaluación permitieron confirmar el comportamiento de los mecanismos implementados, sin que se identificaran errores de ejecución que impidieran completar los escenarios de prueba realizados.

También, durante el desarrollo y las pruebas del proyecto se presentó una dificultad relacionada con la interfaz gráfica, ya que en una versión anterior la ventana del simulador no se visualizaba correctamente. Para solucionar esta situación, se realizaron ajustes en los colores de los componentes visuales, mejorando la presentación y legibilidad de la interfaz.

Después de realizar los cambios, se obtuvo una versión en la que la ventana gráfica pudo visualizarse correctamente



7.5 Conclusión del proceso de evaluación

El proceso de evaluación permitió comprobar el funcionamiento de los principales componentes del simulador de funciones del kernel. Mediante los diferentes escenarios se observó cómo el sistema administra los recursos, organiza la ejecución de procesos, controla las secciones críticas y responde ante situaciones de interbloqueo.

Las pruebas adicionales con procesos aleatorios permitieron verificar el comportamiento del simulador ante solicitudes variadas y confirmar la liberación de memoria después de finalizar los procesos.

Con base en los resultados obtenidos, se considera que el simulador cumple con las funciones evaluadas en los escenarios ejecutados y constituye una representación práctica de los conceptos fundamentales estudiados en Sistemas Operativos.

8. Declaración de Autoría y Honestidad Académica

Los integrantes del equipo, firmantes declaramos que el proyecto titulado “Simulador de Funciones del Kernel de Sistema Operativo” constituye un trabajo original y autónomo, elaborado para satisfacer los requerimientos académicos del curso Sistemas Operativos I.

Certificamos que el trabajo presentado es resultado del esfuerzo y la colaboración de los cuatro integrantes del equipo, y que hemos procurado realizarlo de manera responsable y honesta, respetando las normas de integridad académica.

Asimismo, declaramos que no hemos presentado como propio el trabajo de otras personas asumimos de forma solidaria la plena responsabilidad sobre la originalidad, el rigor técnico y la autoría de este documento, eximiendo a la institución y al cuerpo docente de cualquier controversia derivada del uso indebido de propiedad intelectual o de omisiones en las referencias bibliográficas.

En fe de lo anterior, firmamos en señal de conformidad:

1. **Nombre completo:** Edgar yovany García Luis 7690 23 13075
2. **Nombre completo:** Angel Aurelio Tzi Cucul 7690-23-3711
3. **Nombre completo:** Angel Steve Morejón Martínez 7690-23-21973
4. **Nombre complete:** Dalila Betsabeth García Acabal 7690-23-3074



FIRMA

Fecha de entrega: 03 de Noviembre del 2026

9. Conclusiones y Recomendaciones

9.1 Conclusiones

1. Se cumplió el objetivo general: se construyó un simulador modular del kernel en Java que integra planificación Round Robin, gestión de memoria de 100 MB, semáforo binario y manejo de interbloqueo, operable desde una CLI y una interfaz gráfica.
2. El planificador Round Robin con q = 3 reprodujo exactamente el cálculo teórico: para las ráfagas 7, 4 y 5, los tiempos de espera fueron 9, 9 y 10 unidades (promedio 9.33), lo que valida tanto la implementación como las fórmulas.
3. La asignación contigua dinámica con First-Fit rechazó correctamente los procesos sin hueco suficiente y mostró que el desbordamiento depende del mayor hueco contiguo, no solo de la memoria libre total.
4. La condición de carrera se manifestó sin exclusión mutua (2 de 4 actualizaciones) y desapareció con el semáforo binario (4 de 4), lo que demuestra su necesidad y efectividad.
5. El grafo de espera permitió detectar la espera circular entre dos procesos y dos recursos, y el aborto de un proceso rompió el ciclo y permitió que el otro terminara.
6. La validación centralizada en el Kernel y en la lectura de entradas hace que el simulador resista parámetros anómalos o aleatorios sin detenerse, lo que lo prepara para la prueba de estrés en vivo.
7. Construir el simulador permitió comprender que los mecanismos del sistema operativo no operan aislados: la finalización de un proceso afecta la memoria, el semáforo y los recursos al mismo tiempo, y esa coordinación es responsabilidad del núcleo.

9.2 Recomendaciones

1. Incorporar compactación de memoria o paginación para eliminar la fragmentación externa y comparar el desempeño frente a First-Fit.
2. Agregar otras políticas de asignación (Best-Fit y Worst-Fit) y otros planificadores (FCFS, SJF y prioridades) para contrastarlos con Round Robin bajo las mismas cargas.
3. Permitir tiempos de llegada distintos y un quantum configurable, de modo que el análisis de espera cubra escenarios más realistas.
4. Extender la detección de interbloqueo a múltiples recursos y procesos, e incorporar estrategias de prevención y evitación, como el algoritmo del banquero.
5. Automatizar las pruebas con un marco como JUnit para ejecutar la bitácora de forma continua en cada cambio del repositorio.
6. Utilizar el simulador como material de apoyo didáctico en el curso, apoyándose en la interfaz gráfica para explicar cada mecanismo paso a paso.

Referencias Bibliográficas

- Coffman, E. G., Elphick, M., & Shoshani, A. (1971). System deadlocks. *ACM Computing Surveys, 3*(2), 67–78.
- Dijkstra, E. W. (1965). *Cooperating sequential processes* (EWD123). Technological University Eindhoven.
- Gómez, J. (s. f.). *Documento oficial de enunciado del proyecto final de Sistema Operativo I*. Universidad Mariano Gálvez de Guatemala.
- Oracle. (s. f.). *Java Platform, Standard Edition documentation*. https://docs.oracle.com/en/java/
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.
- Stallings, W. (2018). *Operating systems: Internals and design principles* (9.ª ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern operating systems* (4.ª ed.). Pearson.